The Compliant Wheel Repository

What it does, and the mathematics it runs on

A guided tour of `github/wheel`

Written 2026-08-29 against the `feature` branch

Abstract

A tour of this repository for a reader who is technically fluent but new both to the tree and to structural-optimization mathematics. It explains what the software designs, the specific mathematics each stage implements, the project's private vocabulary, the module architecture, an end-to-end trace of one optimizer step, and the reasoning behind the algorithmically interesting choices. Every claim is traceable to a file and a line; where the code and the plan files disagree, the code is followed. Line references are against the `feature` branch as of 2026-08-29, with `PLAN.md` sections referenced up to §92.

Contents

1 Overview

1.1 The thing being designed

This repository designs *one physical object*: a compliant (deliberately springy) wheel for a small UAV, printed in PLA on a fused-filament (FFF) printer. It is a flat, planar part extruded to a constant face width—you could cut it out of a 22.4 mm thick plate. Its geometry is fixed by a handful of hard requirements (`src/wheel_fea.py:107-165`):

Quantity	Value	Where
outer diameter	⌀100 mm (`RIM_OUTER_RADIUS_MM = 50.0`)	`wheel_wheel.py:173`
hub bore radius	12.7 mm (a $\frac{1}{2}''$ shaft)	`wheel_fea.py:110`
spokes	12, twelve-fold rotationally symmetric	`wheel_fea.py:109`
face width	22.4 mm	`wheel_fea.py:145`
material	PLA, $E = 2300$ MPa, $\nu = 0.35$	`wheel_fea.py:148`
service load	15 lbf = 66.72 N	`wheel_fea.py:152-153`
target axle drop	**2.0 mm**	`wheel_fea.py:158`
allowable stress	25 MPa (50 ult. × 0.80 FFF ÷ 1.6 SF)	`wheel_fea.py:149-151`

Table 1: The fixed requirements. Everything else is optimized.

The interesting entry is the target deflection. This is not a stiff wheel that must not bend; it is a *compliant mechanism*—the 2.0 mm of travel under load **is the feature**, a suspension made of geometry instead of springs. That is why the deflection objective is two-sided: being too stiff is penalised exactly as hard as being too soft (`wheel_fea.py:748-752`, `wheel_objective.py:1152-1158`).

Between the hub and the rim run twelve identical curved spokes. Their shape is what gets optimized.

1.2 What the software actually does

The repository is a *design-optimization pipeline with a verification apparatus bolted to every joint.* It has three stages plus an exporter.

```
Stage 1/2  GENETIC ALGORITHM over an analytical beam model
           wheel_fea.py + pygad          ~minutes         14-gene genome
                 |
                 |   best_solution.json / stage2_elites.json
                 v
Stage 3    GRADIENT DESCENT over a full finite-element contact model
           wheel_stage3.py -> wheel_objective.py -> wheel_adjoint.py
           -> wheel_fem.py -> wheel_wheel.py               ~hours
                 |
                 |   best_solution.json (promoted)
                 v
EXPORT     watertight B-rep solid, filleted, in STEP
           wheel_step_export.py (CadQuery/OpenCascade)      ~minutes
                 |
                 v
           export/wheel.step + wheel_step_manifest.json
```

The two search stages exist because they answer different questions at different prices. The beam model in `wheel_fea.py` scores one design in about 0.25 ms, which is what makes a 300-population × 300-generation genetic search possible. The finite-element model in `wheel_fem.py` scores one design in roughly 18 seconds per load phase, eight phases per evaluation—but it can see things the beam model is structurally blind to.

The single most important measured finding in the tree is that **the beam model is an accurate spoke model and a blind wheel model**: four GA winners whose losses agree to 1.2%

and whose beam deflections agree to 0.5% have real axle drops of 0.71, 1.67, 1.86 and 2.69 mm—a factor of 3.8 (`wheel_wheel.py:522-548`). The mechanism is named there too: the beam model integrates the spoke over its whole hub-to-rim span, but the built part is welded into its rings over two arcs, so only the middle 67–87% of the spoke is a flexure at all.

1.3 Why the repository looks the way it does

Three structural oddities will hit you immediately, and all three are deliberate.

Two virtual environments. `.venv-opt` (numpy/scipy/JAX/pygad) runs the optimizer and the FEA; `.venv-cad` (CadQuery/OpenCascade) runs the STEP exporter. CadQuery pins an incompatible OCC stack and cannot coexist with the rest (`requirements-cad.txt`). The two interpreters share **only source files and JSON**. This forces a hard constraint enforced by `tests/test_import_hygiene.py`: `wheel_fea.py`, `wheel_geometry.py` and `project_paths.py` must import with numpy alone—no JAX at module scope—because the CAD environment imports them across the interpreter boundary (`wheel_step_export.py:71-84`). `wheel_geometry.py` solves this by taking its array module as an argument (`xp=np` or `xp=jax.numpy`) rather than importing one.

Extraordinary comment density. A docstring in this tree is not documentation, it is *evidence*. `wheel_objective.py:165-297` is 130 lines of comment justifying three floating-point constants, including the measurement that falsified the previous values. `CLAUDE.md` explicitly carves out an exception to its own simplicity rule for exactly this reason: "a docstring here carries the measurement that justifies a constant, and deleting it loses evidence, not verbosity."

The plan files. `PLAN.md` is 778 KB and is the project record, organised into numbered sections (§1 … §92). Eight sibling `*_PLAN.md` files are open work arcs. Comments across the tree cite these by number. Six closed arc files were deleted on 2026-08-16 and about seventy citations to them remain dangling on purpose—`PLAN.md:53-77` is the redirect table.

2 Mathematical foundations

This section teaches the specific mathematics the code implements, in the order the data flows. Notation is introduced as it appears.

2.1 The genome: what a design *is*

A design is a point in $\mathbb{R}^{14}$. The gene names, in a fixed order that is a hard contract (`wheel_genome.py:29-36`):

```
cx1 cy1 cx2 cy2 cx3 cy3 cx4 cy4     four interior Bezier control points (mm)
t0  t1  t2  t3                       four thickness nodes (mm)
R_hub  R_rim                         two fillet radii (mm)
```

Each gene has a box constraint (`wheel_fea.py:261-310`). Stage 3 works in a *normalised unit box* $z \in [0,1]^{14}$ rather than in millimetres, because `cy*` spans 64 mm while `R_rim` spans 2.5 mm and one gradient step length cannot mean anything in raw units. The map is affine (`wheel_genome.py:76-88`):

$$z = \frac{g - \ell}{h - \ell} \qquad \texttt{(normalize)} \tag{1}$$

$$g = \ell + z\,(h - \ell) \qquad \texttt{(denormalize)} \tag{2}$$

with ℓ and h the per-gene bounds, and the chain rule $\partial L/\partial z = (\partial L/\partial g) \odot (h - \ell)$ lives in exactly one place, `wheel_objective.objective` (`wheel_objective.py:1376-1379`).

Provenance is enforced structurally. `genome_hash` (`wheel_genome.py:118`) is

$$\text{sha256}(\texttt{json.dumps(sorted(genes.items()))})[:7],$$

so **adding one key inside `genes` changes the hash of every genome ever recorded**. `save_record` raises if `genes` is not exactly the 14 canonical keys; everything else goes into top-level blocks. The shipped genome's hash is `09e8188`.

2.2 Bézier centerlines

Each spoke's centerline is a *degree-5 Bézier curve* in the spoke's own local frame, running from the hub at $(0, 0)$ to the rim at $(S, 0)$, with $S = 48.5 - 12.7 = 35.8\,\text{mm}$.

A Bézier curve of degree n over control points $P_0, \ldots, P_n$ is

$$C(u) \;=\; \sum_{k=0}^{n} B_{n,k}(u)\, P_k, \qquad B_{n,k}(u) \;=\; \binom{n}{k} (1-u)^{\,n-k}\, u^k, \tag{3}$$

where the $B_{n,k}$ are the *Bernstein basis polynomials*. Two facts matter here.

1. **Linearity in the control points.** For a fixed sample grid $u = \text{linspace}(0, 1, N)$ the curve is curve $= B\,P$ with B a constant $(N, 6)$ matrix. That is why the gradient path from genes to node positions is cheap—the map is a matrix multiply. The basis is `lru_cached` because it depends only on N, which removed a measured 21% of a fitness evaluation (`wheel_geometry.py:83-97`).
2. **Endpoint tangency.** The derivative curve (the *hodograph*) of a degree-n Bézier is a degree-$(n-1)$ Bézier over the scaled forward differences $n\,\Delta P_k$. At $u = 0$ this gives $C'(0) = n\,(P_1 - P_0)$, so **the endpoint tangent is exactly the first control-polygon edge**. Since P_0 is locked at the origin, the hub tangent direction is exactly $(\texttt{cx1}, \texttt{cy1})$. `wheel_geometry.forward_difference_matrix` (`:117`) builds the difference operator and `bezier_tangent` (`:148`) applies it. This closed form is what makes the `arrival` barrier free to evaluate (§2.5) and is what `studies/study_arrival_cap.py` exploits to sweep one angle while moving no other gene.

`bezier_curvature` (`wheel_geometry.py:164`) gives the signed curvature analytically,

$$\kappa(u) \;=\; \frac{x'y'' - y'x''}{\left(x'^2 + y'^2\right)^{3/2}}, \tag{4}$$

from the first and second hodographs. This is needed for the fold constraint below.

2.3 The offset band: turning a curve into a part

The spoke is the centerline *thickened along its normal by a variable amount*. Define normalised arc length $s \in [0, 1]$ and a transverse coordinate $\eta \in [-1, +1]$. Then

$$x(s, \eta) \;=\; C(s) \;+\; \eta\, \frac{t(s)}{2}\, \hat{N}(s), \tag{5}$$

where $\hat{N}$ is the unit normal (the tangent rotated $+90°$) and $t(s)$ is the local thickness. This one formula is `wheel_geometry.offset_band` (`:290`), and it does double duty:

- at `n_across = 1` it returns the two flanks—the outline the STEP exporter builds from;
- at `n_across = 6` it returns a structured quadrilateral mesh block.

The meshed part and the exported part are therefore the same surface by construction, not by two pipelines agreeing (`wheel_mesh.py:14-21`).

Thickness is piecewise-linear over three equal arc-length zones with nodes $t_0 \dots t_3$ at $s = 0, \frac{1}{3}, \frac{2}{3}, 1$. The original numpy implementation used boolean masks; JAX has no equivalent, because `s[mask]` has a data-dependent shape. `thickness_at_arc_length` (`wheel_geometry.py:217`) rewrites it branch-free as a base value plus three clipped ramps:

$$t(s) \;=\; t_0 \;+\; \sum_{i=0}^{2} (t_{i+1} - t_i)\;\mathrm{clip}\!\left(\frac{s - b_i}{b_{i+1} - b_i},\, 0,\, 1\right). \tag{6}$$

The equivalence argument is in the docstring: inside zone i, ramps $j < i$ are saturated at 1 and telescope to $t_i - t_0$, ramp i contributes the interpolation fraction, and ramps $j > i$ clip to 0. This is exactly `np.interp`, and the docstring records that dropping the original's spurious $+10^{-12}$ made the function hit its own interpolation nodes exactly.

2.4 The fold constraint: where an offset curve destroys itself

This is the first piece of genuinely non-obvious geometry.

If you offset a curve outward by a distance d, and the curve's local *radius of curvature* $\rho = 1/|\kappa|$ drops below d, the offset passes *through the centre of curvature* and turns inside out. The flank crosses itself; the region has negative area; the corresponding mesh element is inverted; the STEP wire self-intersects. The part described by such a genome **does not exist**, and yet the Castigliano beam model will happily return a deflection for it.

The signed clearance is (`wheel_geometry.self_intersection_margin`, `:421`):

$$\text{margin} \;=\; \min_{s} \left(\frac{1}{|\kappa(s)|} \;-\; \frac{t(s)}{2} \right). \tag{7}$$

Negative means folded. This is exposed as a *smooth signed scalar* precisely so it can serve as a barrier term rather than be discovered as a crash.

The threshold is not zero, and the reason is measured (`wheel_geometry.py:355-374`). Over 2001 feasible genomes sampled by `studies/study_mesh_quality.py`:

margin >	missed bad meshes	false alarms	designs kept
0.00	6	87	1187
0.05	2	116	1154
0.10	**0**	140	1128
0.20	0	196	1072

Table 2: Why `MIN_FOLD_MARGIN_MM` is 0.1 and not 0.

`MIN_FOLD_MARGIN_MM = 0.1` is the smallest threshold with zero misses, and it keeps 95% of the designs a zero threshold would. Roughly **40% of the genomes the other constraints admit** have a folded region (`wheel_fea.py:653-671`)—this is not an exotic failure.

2.5 The arrival angle, and why its sense is backwards

The centerline endpoints are *locked* on the hub and rim circles, and the end cross-section is normal to the centerline. Define the *arrival angle* as the angle between the centerline and the ring's *tangent* at that endpoint: 0° tangential, 90° radial. Closed form (`wheel_wheel.arrival_angles`, `:380`), using the hodograph endpoint fact from §2.2 and the fact that the endpoint's own unit vector *is* the local radial direction:

$$\text{arrival} \;=\; \arcsin\!\left|\, \hat{d} \cdot \hat{r} \,\right|. \tag{8}$$

The intuition is that a near-*tangent* arrival is the hard case—and for the CAD fillets it is. For the *mesh* it is exactly backwards. Because the end cross-section is normal to the centerline, a

near-tangent arrival makes the cross-section nearly *radial*, so it meets the ring arc at about 80° and the junction block is beautifully shaped. A near-*radial* arrival lays the cross-section *along* the arc and the corner collapses. Measured over 58 feasible genomes, the correlation between arrival angle and junction minimum scaled Jacobian is −0.93 at the hub and −0.91 at the rim:

arrival (deg)	4.9	7.2	64.8	82.2
min. scaled Jacobian	0.806	0.804	0.415	0.043

Past about 80° both flanks lie *outside* the ring circle and the spoke touches its ring at a single point. That is a **hinge, not a weld**—and since the beam model assumes a moment connection at both ends (`BOUNDARY_CONDITION = "fixed_guided"`), it would be solving a structure the part does not have. The threshold sweep over 200 genomes is sharp: the worst genome failing minSJ > 0.2 arrives at 70.6°, and every threshold at or below 70° gives zero misses. `MAX_ARRIVAL_DEG = 65.0` takes 5.6° of margin (`wheel_wheel.py:355-377`).

2.6 Barriers: how constraints become part of a smooth objective

Every constraint in this tree is a *quadratic soft barrier* (`wheel_fea.py:579`; the `jnp` version at `wheel_objective.py:414`):

$$\text{soft\_barrier}(v, w) \;=\; w \cdot \max(0, v)^2. \tag{9}$$

Zero when satisfied, rising quadratically when violated. Quadratic rather than linear so that it is C^1 at the knee—which is exactly what a finite-difference plateau needs, and what a gradient-based line search needs in order to avoid a kink. This replaced a hard −500 cliff that a gradient-free search could not descend.

The repository is careful about one distinction that is *not* derivable from the weight table (`wheel_objective.py:394-407`):

- a **barrier term** answers "may this design ship?"—its only admissible value is 0, and it never trades against mass;
- an **objective term** answers "how good is it?"

Conflating them caused a real defect: a run reported its lowest-*loss* iterate, which was one that violated the hub fillet cap slightly and bought a lot of mass for it. Fifty-five of its 101 iterates were feasible and it promoted an infeasible one. The fix is `wheel_stage3.selection_key` (`:195`), a three-tier lexicographic sort described in §6.7.

2.7 The beam model: the force method (Castigliano)

Stage 1/2's physics is `wheel_fea.generalized_spoke_mechanics` (`:398`). It solves one curved spoke as a *statically indeterminate* beam by the force method.

Set the local frame with the hub root at $(0, 0)$ and the rim tip at $(S, 0)$. The radial service load F acts at the tip along $-x$. If the tip were free, the internal moment and axial force at a station would be determined by statics alone. It is not free: the spoke is fused into a stiff rim ring, which restrains tip rotation and tangential motion. So two *redundants* are carried at the tip:

- M_0 — an end moment, restraining tip rotation;
- H — a tangential force, restraining tip motion along $+y$.

Cutting a free body from station s to the tip gives the internal actions

$$M(s) = M_0 + H\,(S - x) - F\,y, \tag{10}$$

$$N(s) = -F\cos\theta + H\sin\theta, \qquad (\cos\theta, \sin\theta) = \left(\tfrac{dx}{ds}, \tfrac{dy}{ds}\right), \tag{11}$$

with N **negative in compression** (v2.0 had this backwards, which is why its buckling check never fired). Write these as linear combinations of unit-load fields:

$$M = M_0\, m_{M_0} + H\, m_H + F\, m_F, \qquad m_{M_0} = 1, \qquad m_H = S - x, \qquad m_F = -y, \tag{12}$$

$$N = M_0\, n_{M_0} + H\, n_H + F\, n_F, \qquad n_{M_0} = 0, \qquad n_H = \sin\theta, \qquad n_F = -\cos\theta. \tag{13}$$

The *complementary strain energy* of a beam carrying moment and axial force is

$$U \;=\; \int \frac{M^2}{2EI}\, ds \;+\; \int \frac{N^2}{2EA}\, ds. \tag{14}$$

Castigliano's second theorem says $\partial U/\partial X = 0$ for any redundant X whose corresponding displacement is restrained to zero. Applying it to M_0 and H gives a 2×2 linear system in the *flexibility coefficients*

$$a_{ij} = \int \frac{m_i m_j}{EI}\, ds + \int \frac{n_i n_j}{EA}\, ds, \qquad b_i = \int \frac{m_i m_F}{EI}\, ds + \int \frac{n_i n_F}{EA}\, ds, \tag{15}$$

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{12} & a_{22} \end{bmatrix} \begin{bmatrix} M_0 \\ H \end{bmatrix} \;=\; -F \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}, \tag{16}$$

which the code assembles as segment sums over the 600-point discretization (`wheel_fea.py:495-520`). A tiny relative ridge $+10^{-12}(1 + |a_{ii}|)$ is added to the diagonal, because a near-straight, very thin spoke makes the system ill-conditioned and a genetic algorithm will propose exactly that.

The tip deflection along the load direction is then Castigliano's *first* application, $\partial U/\partial F$:

$$\delta \;=\; \left| \sum \frac{M\, m_F}{EI}\, \Delta s \;+\; \sum \frac{N\, n_F}{EA}\, \Delta s \right|. \tag{17}$$

The comment at `wheel_fea.py:522-525` makes an argument worth understanding: by the *envelope theorem*, the redundants sit at stationary values of U, so differentiating with respect to F while *holding them fixed* is exact—the chain terms through dM_0/dF and dH/dF vanish. No extra work is needed.

The legacy `bc="cantilever"` sets $M_0 = H = 0$, which collapses algebraically to $M = -Fy$ and $N = -F\cos\theta$—term for term the v2.0 integrands. That equivalence is the regression test. The physical consequence of the change: for a straight beam, cantilever gives $FL^3/3EI$ and fixed-guided gives $FL^3/12EI$, so **v2.0 over-predicted compliance by a factor of four**.

Three corrections ride on top.

- **Peterson stress concentration** (`wheel_fea.py:360`). At a fillet in a stepped beam in bending, $K_t \approx 1 + C\,(t/2R)^{0.65}$, clamped to $[1.0, 3.5]$. The peak bending stress at each end is multiplied by its local K_t. This same formula reappears—differentiated rather than frozen—as the FEA-era stress constraint (§2.14).

- **Winkler–Bach curved-beam correction** (`wheel_fea.py:376`). On a curved member the neutral axis shifts toward the centre of curvature and inner-fibre stress is higher than straight-beam theory says. Simplified: $k_b = 1 + h/(4R)$, clipped to $[1.0, 2.5]$, significant only when $R < 4h$.

- **Euler buckling as an equivalent column** (`wheel_fea.py:543-556`):

$$P_e \;=\; \frac{\pi^2\, EI_{\text{eff}}}{(K_e L)^2}, \qquad EI_{\text{eff}} \;=\; \frac{L}{\sum \Delta s/EI}, \tag{18}$$

  where EI_{eff} is the *compliance-weighted (harmonic) mean* rigidity—the standard equivalent-column treatment for a tapered member. v2.0 applied this per discretization segment, where $L \approx 0.06\,\text{mm}$ made $1/L^2$ astronomical and the check could never fire. The docstring is honest that for an in-plane-curved member Euler is a proxy: the genuinely competing modes are snap-through and lateral-torsional buckling.

2.8 The genetic algorithm

`wheel_fea.evaluate_design` (`:699`) sums nine weighted terms into a scalar loss; `pygad_fitness` returns its negation because PyGAD maximises. The GA itself (`wheel_fea.py:1310` ff.) is population 300 (at least 20× the gene count), tournament selection with $k = 5$, uniform crossover, elitism 5, and an *adaptive Gaussian mutation* whose σ starts at 0.25 of the gene range and halves halfway through the run (`adaptive_gaussian_mutation`, `:881`).

Two of the loss terms are worth naming, because they are the *mesh's* constraints imported into a search that predates the mesh: `fold` (§2.4) and `arrival` (§2.5), both closed-form and therefore free next to the Castigliano solve, and both fixing cases where **the model itself is invalid** rather than merely where the design is poor.

2.9 Finite elements: write the energy, derive everything else

`wheel_fem.py` is the FEA kernel, and its organizing principle is stated in the first paragraph of its docstring: **`element_energy` is the only physics in the file.** Internal force and tangent stiffness are its gradient and Hessian, obtained by `jax.grad` and `jax.hessian` and `vmap`ped over elements (`wheel_fem.py:258-278`):

$$f_e \;=\; \nabla_u \,\Pi_e, \qquad K_e \;=\; \nabla_u^2 \,\Pi_e. \tag{19}$$

There is no B-matrix, no hand-differentiated stress recovery, and no separately coded tangent that can drift out of step with the residual. That entire class of bug is absent by construction, and the rigid-body test becomes exact rather than a cancellation: if the energy is frame-indifferent, so is everything derived from it.

Elements. Isoparametric quadrilaterals: Q4 (bilinear, 4 nodes) or Q9 (biquadratic, 9 nodes). Q9 is the production choice because **a Q4 mesh shear-locks in bending**—it reports the structure as stiffer and safer than it is, which is catastrophic for a 2 mm flexure. Full Gauss integration (2×2 for Q4, 3×3 for Q9); reduced integration is the usual dodge for Q4 locking but buys hourglass modes (`wheel_fem.py:144-160`).

The Jacobian, and one index that matters. In `element_energy` (`:235`):

```
J      = jnp.einsum("gnk,ni->gki", dN, Xe)      # J[g,k,i] = dx_i/dxi_k
detJ   = J[:,0,0]*J[:,1,1] - J[:,0,1]*J[:,1,0]
dNdx   = jnp.einsum("gnk,gik->gni", dN, jnp.linalg.inv(J))
grad_u = jnp.einsum("ni,gnj->gij", ue, dNdx)
```

`J` here is the *transpose* of the usual Jacobian matrix, so $\partial \xi_k / \partial x_i$ is `inv(J)[i,k]`, not `inv(J)[k,i]`. Getting it backwards is invisible on an axis-aligned rectangle (J is diagonal) and silently wrong on every curved element—which is exactly the geometry this project meshes. The distorted-mesh patch test is what catches it.

Constitutive law. St. Venant–Kirchhoff strain energy density (`wheel_fem._energy_density`, `:221`):

$$W(\varepsilon) \;=\; \frac{\lambda}{2}\,(\operatorname{tr}\varepsilon)^2 \;+\; \mu\,(\varepsilon : \varepsilon), \tag{20}$$

with Lamé constants from `lame()` (`:190`). *Plane stress* uses the reduced $\lambda = E\nu/(1-\nu^2)$; plane strain uses $\lambda = E\nu/\big((1+\nu)(1-2\nu)\big)$. The choice is deliberate and recorded in the result dictionary: plane stress makes a uniaxial state give $\sigma = E\varepsilon$ exactly, so beam bending gives $EI = Ewt^3/12$ with *plain* E—apples to apples with the Castigliano model, which is what makes the beam-versus-FE comparison a check rather than an argument. The real 22.4 mm-wide part is closer to plane *strain* (effective modulus $E/(1-\nu^2) = 1.14E$), and that 14% is larger than most effects the project chases, so `plane="strain"` exists and is never picked silently (`wheel_fem.py:49-61`).

2.10 Two kinematics, five lines apart

`_strain` (`wheel_fem.py:207`) returns either

$$\text{linear:}\quad \varepsilon = \tfrac{1}{2}(\nabla u + \nabla u^{\mathsf{T}}), \tag{21}$$

$$\text{svk:}\quad E = \tfrac{1}{2}(F^{\mathsf{T}}F - I), \qquad F = \nabla u + I. \tag{22}$$

Green–Lagrange is exactly frame-indifferent. Under a rigid rotation $u = (R - I)x$ we have $F = R$, so $E = \frac{1}{2}(R^{\mathsf{T}}R - I) = 0$ identically: a rigid rotation of *any* magnitude stores zero energy. The linear strain does not have this property, and that is the entire content of the finite-rotation test. Because the two share the same `_energy_density`, they cannot disagree about the material.

There is a trap the file names explicitly (`wheel_fem.py:980-1010`): at $u = 0$ the SVK Hessian *equals* the linear one exactly. So routing an `svk` problem through `solve_linear` returns a linear answer silently. The fix is `solve()` (`:1260`), which dispatches on `prob.nonlinear`.

How much does it matter? `studies/study_gnl.py` measures it: at service load the wheel is **23.16%** softer under SVK than under linear kinematics for the shipped genome—and, more decisively, that correction is *not* a property of the wheel that can be applied once. Held at a fixed axle drop by scaling the load per genome, it still ranges over roughly an order of magnitude across feasible designs. So the nonlinear solve has to be in the loop. `wheel_stage3.py -kinematics` defaults to `svk` since `PLAN` §32; `wheel_fem`'s own kernel defaults stay `linear` on purpose so M4's committed reports stay reproducible.

2.11 Contact: the ground is not a pressure you assume

Early full-wheel work applied the ground load as an assumed elliptical pressure over an assumed 3° patch (`_ground_traction`, `wheel_fem.py:1462`). Two choices there are worth noting: the traction is **vertical, not radial** (for frictionless contact against a rigid *flat* ground the bodies share a tangent plane, so the traction is along the ground's normal—using the rim's radial normal produces a spurious horizontal resultant), and it is **elliptical, not uniform** (a uniform patch has a pressure step at its edges, and a step puts a stress singularity into a model whose peak stress is a reported number).

But the patch half-angle was a free parameter, and sweeping it moved the axle drop by 9.7% over a 12× range—larger than the entire geometric-nonlinearity correction. So M6 replaced it with *penalty contact* against a rigid frictionless ground.

`RigidGroundContact` (`wheel_fem.py:652`) adds a potential to the total energy. For a boundary segment with gap $g = (x_y + u_y) - y_{\text{ground}}$, the penalty potential density is $\varepsilon_n\,\varphi(-g)$ where φ is a smoothed positive part (`_penalty_phi`, `:592`). The sharp branch is $\varphi(z) = \frac{1}{2}\langle z\rangle^2$, which is C^1; the smoothed branch is a cubic matching value, slope *and* curvature at both $z = 0$ and $z = d$, so φ'' is continuous everywhere:

$$\varphi(z) \;=\; \begin{cases} 0, & z \le 0, \\ \dfrac{z^3}{6d}, & 0 < z < d, \\ \dfrac{z^2}{2} - \dfrac{dz}{2} + \dfrac{d^2}{6}, & z \ge d. \end{cases} \tag{23}$$

The same `grad`/`hessian` machinery produces the contact force and contact stiffness, so they cannot disagree with the potential.

The problem is displacement-driven, and that is not a convenience. With a rigid flat ground indenting by δ, the rise of the wheel's lowest point *is* δ, so the axle drop is an **input** and the wheel load is the **output** (`wheel_fem.py:1735-1751`). A force-driven contact problem is singular at

its own starting point—before contact establishes there is no pressure at all. `solve_wheel_contact` (`:1867`) then inverts this with a *secant iteration* to answer the design question ("what is the drop at 66.72 N?"), each solve warm-starting from the previous.

The measured payoff: the real patch is a half-angle of about 0.47°, not 3.0°—the assumed patch was six times too wide—and yet the axle drop moves only 1.3%, because the drop is dominated by spoke and rim bending rather than by how the last few newtons are spread (`studies/study_contact.py:14-30`).

2.12 Constraints by master–slave elimination

`DofMap` (`wheel_fem.py:828`) expresses every boundary condition as one sparse matrix,

$$u_{\text{full}} \;=\; T\,u_{\text{reduced}} \;+\; u_{\text{prescribed}}, \tag{24}$$

and the reduced system $T^{\mathsf{T}}KT$ is symmetric positive definite whenever the full one is. This single mechanism handles homogeneous and inhomogeneous Dirichlet conditions, *skew* constraints (`constrain_direction`: enforce $u\cdot\hat{d}=0$, leaving the perpendicular free—this is how "plane sections remain plane but may stretch laterally" is expressed without rotating the system into a local frame), and **rigid ties**:

$$u_x = U_x - \Theta\,(y - y_c), \tag{25}$$
$$u_y = U_y + \Theta\,(x - x_c), \tag{26}$$

which tie a whole cross-section to a 3-DOF rigid body. That is how the spoke's tip boundary condition is made to mean exactly what beam theory assumes: plane sections remain plane *and normal*. The two redundants of §2.7 reappear here as reactions—M_0 is the reaction of $\Theta=0$, and H the reaction of the transverse constraint.

`finalize()` asserts that **every DOF is claimed exactly once**: an unclaimed DOF is a free floating node (singular matrix), and a doubly-claimed one silently drops the first constraint.

2.13 Newton with an energy line search

`solve_nonlinear` (`wheel_fem.py:1116`) is a Newton–Raphson loop with load continuation and Armijo backtracking. One detail is genuinely instructive:

```
slope = float(r @ du)
if slope >= 0.0:
    raise NewtonDivergedError("...the tangent is not positive definite,
                               i.e. this equilibrium is unstable")
```

Because `r` really *is* $\nabla\Pi$ rather than an approximation of it, `slope` is exact. So a non-negative value is not line-search noise—it says the tangent lost positive definiteness, i.e. **this equilibrium is unstable**. Saying that is far more useful than letting the backtrack loop grind α down to nothing. This is also, as `wheel_stage3.py:76-85` notes, the only buckling signal the repository owns until the Euler proxy is replaced.

Convergence fires on *either* the relative reduced residual $\|r\|/\|f_{\text{ref}}\| \le \text{tol}$ *or* the energy increment $|du\cdot r|$ falling to $\text{tol}_{\text{energy}}$ of the first step's — the second because at low load it is the only criterion attainable.

2.14 Stress: why the maximum is not a number

The peak von Mises stress in this wheel **diverges under mesh refinement**: 31.02 → 41.54 → 48.47 MPa over coarse/medium/fine. That is not a solver defect. The spoke/ring junction is a

re-entrant corner, and the elastic field at a traction-free wedge of material angle ω behaves as $\sigma \sim r^{\lambda-1}$ where λ is the smallest root in $(0,1)$ of the *Williams eigenvalue equation* (mode I):

$$\sin(\lambda\omega) \;+\; \lambda\,\sin(\omega) \;=\; 0. \tag{27}$$

`studies/study_corner_singularity.py:132` solves this by bisection and verifies it against the two cases with known answers: $\omega = 360°$ (a crack) gives exactly 0.5, and $\omega = 270°$ gives the textbook 0.5445. The measured junction wedges here are 295°–356°, all re-entrant, so $\lambda - 1 < 0$ and the stress is unbounded at the corner. **The maximum is a property of the mesh, not of the wheel.**

This has three consequences the code carries.

(a) The optimizer cannot use `max`. Its derivative is a delta on whichever Gauss point happens to be highest, so it jumps discontinuously the moment the argmax moves—which, on a rolling wheel with a contact patch sweeping the rim, is every few steps. The smooth surrogate is a *volume-weighted p-norm* (`wheel_adjoint._qoi_pnorm_stress`, `:313`):

$$\sigma_{pn} \;=\; \left(\frac{\sum_g V_g\,\mathrm{vm}_g^{\,p}}{\sum_g V_g}\right)^{1/p}, \qquad V_g \;=\; w_g\,\det J_g\,\cdot\,\text{width}. \tag{28}$$

Volume weighting is essential: an unweighted p-norm scores the *mesh*, because refining the rim adds Gauss points in a lightly stressed region and drags the average down—the optimizer could reduce the objective by changing `cfg`. With volume weights the sum is a quadrature of an integral and is mesh-convergent.

(b) Rescaling a p-norm to the maximum cannot work. The constraint used to be $c\,\sigma_{pn} \le$ allowable with $c = \max/\sigma_{pn}$ measured and frozen within a step. The freeze was mathematically sound—the p-norm is positively homogeneous of degree 1, so $d(c\,\sigma_{pn}) = c\,d(\sigma_{pn})$ *if and only if* c did not move—but the quantity was not. `make m8bi6` swept ten exponents off one set of solves and found the two factors converge in **disjoint ranges of** p: the p-norm for $p \le 6$, the ratio c only for $p \ge 24$, and their product at *no* exponent at either design. Here c is anchored to a divergent quantity, and a frozen divergent number is still a divergent number.

(c) So the peak is modelled, not measured. The shipped constraint (`wheel_objective.py:1160-1200`) is

$$K_t(R,t)\;\cdot\;\sigma_{\text{nominal}}(p{=}4) \;\le\; \sigma_{\text{allow}} \;=\; 25.0\ \text{MPa}, \tag{29}$$

where σ_{allow} is `ALLOWABLE_STRESS_MPA`. It is one `soft_barrier` per junction, summed—the hub priced on (`R_hub`, t_0), the rim on (`R_rim`, t_3). Here σ_{nominal} at $p = 4$ is mesh-convergent (GCI 0.45% / 0.20%, observed order 1.76 / 2.18, comparable to the axle drop's 2.44 / 0.14%), and $K_t = 1 + C(t/2R)^{0.65}$ is the same Peterson factor from §2.7, now **differentiated by `jax.grad` rather than frozen**. Two junctions get two barriers rather than one aggregated K_t, because a hard `max` over the two would zero the gradient of whichever is not currently worst and reintroduce exactly the argmax kink the p-norm exists to blur.

Note the consequence: `STRESS_NOMINAL_P = 4.0` (`wheel_objective.py:147`) is the constraint's exponent, while `wheel_adjoint.STRESS_PNORM_P = 30.0` (`:283`) stays the adjoint's documented default so historical records keep their meaning. Two exponents, two meanings, both pinned by tests.

2.15 Mesh convergence: Richardson and the GCI

Because several key quantities are only meaningful in the limit, the tree does formal mesh-convergence analysis. Given three solutions ϕ_1 (coarse) to ϕ_3 (fine) at cell sizes $h_1 > h_2 > h_3$, Roache's *observed order of convergence* solves

$$p \;=\; \frac{\left|\ln\left|e_{32}/e_{21}\right| + q(p)\,\right|}{\ln r_{21}}, \qquad q(p) \;=\; \ln\!\left(\frac{r_{21}^{\,p} - s}{r_{32}^{\,p} - s}\right), \tag{30}$$

by fixed-point iteration from $q = 0$ (`studies/study_deflection_gci.py:128`). The *Richardson extrapolation* to $h \to 0$ and the *Grid Convergence Index* (a conservative uncertainty band on the extrapolated value) follow at `:166`. The docstring records a real bug: the first draft used the coarse-pair ratio as the denominator, because Roache indexes 1 as the *finest* grid while `phi` here runs coarse to fine.

A caution the tree learned the hard way: **look at the increment ratios before quoting a ladder.** `studies/study_knee_rungs.py` shows a five-rung ladder whose ratio *climbs toward 1* instead of decaying, so every extrapolation lands above the knee while every measurement lands below it. A rising ratio means the top rung is a lower bound, not an estimate.

2.16 The adjoint method

This is the mathematical centrepiece of Stage 3, and `wheel_adjoint.py`'s module docstring is the derivation. The setup is a converged contact equilibrium with

$$\Pi(c, u_f, y) = U_{\text{bulk}}(c, u_f) + \Pi_{\text{contact}}(c, u_f, y), \tag{31}$$

$$u_f = T u_r + u_{\text{pre}} \qquad (T \text{ static: topology frozen}), \tag{32}$$

$$r(c, u_r) = T^{\mathsf{T}} \nabla_{u_f} \Pi \;=\; \nabla_{u_r} \Pi \;=\; 0 \qquad \text{at the solution}, \tag{33}$$

$$K_r = \nabla^2_{u_r} \Pi \qquad \text{what Newton already assembled}, \tag{34}$$

where c is the node coordinates and y the ground position. For a quantity of interest $Q(c, u_f, y)$, differentiate the equilibrium condition $r = 0$ implicitly. The result is

$$K_r \,\text{adj}_r = -T^{\mathsf{T}} \frac{\partial Q}{\partial u_f} \qquad \text{one sparse solve; } K_r \text{ is symmetric}, \tag{35}$$

$$\text{adj}_f = T \,\text{adj}_r, \tag{36}$$

$$\frac{dQ}{dc} = \left.\frac{\partial Q}{\partial c}\right|_u + \nabla_c\Big[\,\text{adj}_f \cdot \nabla_{u_f} \Pi(c, u_f, y)\,\Big], \tag{37}$$

$$\frac{dQ}{d\,\text{genes}} = \text{vjp}_{\texttt{mesh\_coords}}\!\left(\frac{dQ}{dc}\right). \tag{38}$$

Equation (37) **is the whole adjoint: one `jax.grad` of one scalar.** There is no separately coded sensitivity operator, for the same reason there is no separately coded contact stiffness. The cost is one sparse back-solve per quantity, sharing one factorisation (`adjoint_grads`, `wheel_adjoint.py:516`, uses `scipy.sparse.linalg.factorized` and reuses the `jax.vjp` closure across quantities).

Two things *fall out* rather than being coded:

- The total contact force is **exactly** $+\partial\Pi/\partial y_{\text{ground}}$ — the penalty potential's derivative with respect to the ground's position *is* the resultant it presses with. So `_qoi_contact_force` is not a reimplementation of the pressure quadrature, and the two agreeing is a free correctness check.
- $d(\text{force})/d(\text{indentation})$ is the same adjoint with the same multiplier, differentiated with respect to y_{ground} instead of c.

Load control, done as an implicit function. Stage 3 loads to a *force*, not an indentation, so every quantity is evaluated at $\delta^*(p)$ solving $F(\delta^*, p) = F_{\text{service}}$. The naive move—differentiate the secant iteration—would put the secant's *stopping tolerance* into the answer, which is a property of the stopping rule and not of the wheel. Instead the quotient rule for implicit functions is used directly (`service_qoi_value_and_grad`, `wheel_adjoint.py:645`):

$$\frac{d\delta^*}{dp} = -\frac{(\partial F/\partial p)\big|_\delta}{(\partial F/\partial \delta)\big|_p}, \tag{39}$$

$$\left.\frac{dQ}{dp}\right|_{\text{total}} = \left.\frac{\partial Q}{\partial p}\right|_\delta + \frac{\partial Q}{\partial \delta}\cdot\frac{d\delta^*}{dp}. \tag{40}$$

Numerator and denominator both come from the single `contact_force` adjoint at the converged state. The coupling term is **not a correction**: measured at the shipped genome, dropping it does not lengthen the stress gradient, it *reverses* it (+11.22 against a true −2.59). Keeping only the frozen term leaves a gradient with the right direction and the wrong length—and no line search reports that as an error; it reports it as a hard problem.

There is also a guard worth quoting, because it encodes physics as a sanity check:

```
if not np.isfinite(dF_ddelta) or dF_ddelta <= 0.0:
    raise RuntimeError("a wheel that gets softer the harder it is pressed
                        is not a load case, it is a solver failure")
```

2.17 Structured multi-block meshing

The mesh is not produced by an external mesher, and that is a load-bearing decision: running `gmsh` would put a combinatorial, non-differentiable step in the middle of the gradient path, and remeshing between optimizer iterations would make the objective discontinuous (`wheel_mesh.py:22-27`). Instead:

> **Node positions are a closed-form differentiable function of the 14 genes; element connectivity is a compile-time constant.**

Why a clean partition exists at all. The spoke is a spiral sweeping about 45° of angle inside a 30° sector, so the first guess is that adjacent spokes must overlap. Three measured facts save it (`wheel_wheel.py:14-38`):

1. **Radius is strictly monotone** along the centerline (12.700 → 48.900 in the docstring's own numbers, which predate the rim band being thickened inward to `RIM_RADIUS_MM = 48.5`; the argument is unchanged) while θ rises to 45.16° and returns. So each spoke is a single-valued curve $\theta(r)$, and twelve rotated copies of a single-valued $\theta(r)$ can never intersect. Minimum clearance outside the hub circle: 0.891 mm.
2. The **weld footprint** on each ring circle is smaller than the sector—15.85° at the hub and 13.18° at the rim, of 30 available—so the twelve junctions tile each ring with a genuine gap.
3. The centerline endpoints are **locked** on the ring circles, and the end cross-section is symmetric about the centerline, so it crosses its ring circle exactly at its own midpoint. That corner is available in closed form with *no root-find at all*.

Fact 1 makes a structured mesh possible; facts 2 and 3 make the junction blocks well-shaped instead of slivers.

Seven blocks per sector, twelve sectors (`BLOCK_ORDER`, `wheel_wheel.py:2394`): `spoke`, `hub_junction`, `rim_junction`, `hub_collar_weld`, `hub_collar_free`, `rim_band_weld`, `rim_band_free`. The two ring blocks per ring are *split at the weld footprint* rather than graded—because splitting makes the weld arc's two ends into block *corners*, so a contiguous run of ring nodes coincides with the junction's arc nodes by construction rather than by arranging a distribution to land on them.

Coons patches. The junction blocks are four-sided curvilinear regions, filled by a *bilinearly blended Coons patch* (`coons_patch`, `wheel_wheel.py:611`). Given the four boundary curves,

$$\begin{aligned} X(u,v) \;&= \big[(1-v)\,\mathrm{bottom}(u) + v\,\mathrm{top}(u)\big] && \text{ruled in } v \\ &\quad + \big[(1-u)\,\mathrm{left}(v) + u\,\mathrm{right}(v)\big] && \text{ruled in } u \\ &\quad - \big[\text{bilinear interpolation of the four corners}\big]. \end{aligned} \tag{41}$$

The subtraction removes the double-counting of the corners; the result interpolates all four boundaries exactly. This is the classic transfinite interpolation construction. The code checks corner agreement to 10^{-9} mm on the numpy path, because a swapped or unreversed edge produces a patch that is *folded* rather than merely ugly, and the fold surfaces as a negative Jacobian a long way downstream.

Ring stations, and a two-stage root find (`ring_station`, `wheel_wheel.py:296`). Where a flank crosses its ring circle must be exact, because that point is a corner shared by three blocks. Stage 1 brackets by finding an actual *sign change* of $r(s)$ − radius on a dense grid—deliberately *not* by globally inverting $r(s)$, because an S-shaped centerline can cross the circle three times and a global `interp` lands on the wrong branch (measured: it left the Coons corner 5×10^{-3} to 2×10^{-2} mm out on 2 of 60 feasible genomes). Stage 2 is a *fixed count of Newton refinements* against the sampler itself, bringing $|x|$ = radius to 10^{-15}. The fixed count is what keeps it traceable *and* differentiable: unrolled Newton's derivative converges to the implicit-function derivative, so no custom JVP rule is needed.

Seams and single ownership. This is the safety net of the whole module:

> A seam mismatch produces a mesh that plots correctly, has positive Jacobian everywhere, and quietly models a wheel with twelve cracks in it. Nothing about the solve complains.

So every block writes its own coordinates independently, a *union-find* structure (`_UnionFind`, `:2456`, with path compression and lowest-id-wins so ownership is deterministic) merges the node ids declared equal, and `check_seams` reports the largest distance between what an owner and a non-owner *independently computed* for the same node. That number must be at machine precision—measured 3×10^{-14} mm for the filleted sector, 7.1×10^{-15} for the tri-block.

Element orientation. A polar block indexed (θ, r) is left-handed, because $\hat{r} \times \hat{\theta} = +\hat{z}$. Its elements come out with negative Jacobian, which in an FE assembly is not merely cosmetic—it contributes *negative stiffness*, which looks to an optimizer like free compliance. `_orient_elements` (`:2971`) flips whole blocks rather than trusting each one.

Mesh quality: the scaled Jacobian. `wheel_mesh.scaled_jacobian` (`:323`) computes, at each of the four corners of an element, the cross product of the two edges meeting there normalised by their lengths—i.e. **the sine of the corner angle**—and takes the worst. A value of 1.0 is a perfect rectangle; near 0 is a collapsed sliver; **negative is inverted**. It deliberately measures *shape*, not aspect ratio (a long thin rectangle scores 1.0), which is the right property for validity. Its `xp=jax.numpy` path is what makes the mesh-validity barrier differentiable.

2.18 Phase averaging and randomized quasi-Monte Carlo

A rolling wheel presents a different spoke pattern to the ground at every angle. The pattern period is `SECTOR_DEG` $= 30°$. Measured, the axle drop varies **7.0% std/mean and 19.7% peak-to-peak** over that period—so phase is a design variable, not a quadrature nuisance, and `phase_ripple` is a real objective term.

`phase_stencil` (`wheel_objective.py:966`) offers three schemes. The default is `rqmc`: an 8-point uniform stencil shifted by a random offset *drawn from an 8-point sub-lattice*. This is a quantized randomized-QMC design, and the quantization is a performance fact rather than statistical taste.

- *Randomizing at all* matters because the rim's contact faceting lives at $0.25°$ while the stencil samples at $3.75°$, so the artefact **aliases**. Under a fixed stencil the aliased artefact is a deterministic function of the genes with a nonzero gradient—precisely what an optimizer chases. A random shift makes it zero-mean noise instead.
- *Quantizing* matters because `wheel_wheel.coord_fn` keys its JIT cache on `float(phase)`. A *continuously* random offset misses on every phase of every step and pays a measured 0.774 s re-trace eight times per step—roughly doubling the actual solving. The union of all reachable phases is a fixed $8 \times 8 = 64$-point lattice, and `_COORD_FN_CACHE_MAX = 128` is sized to hold it.

2.19 Projected Adam

Stage 3's optimizer is Adam with a cosine learning-rate schedule, global gradient-norm clipping, and *projection* onto the unit box, $z \leftarrow \mathrm{clip}(z - \mathrm{step}, 0, 1)$ (`wheel_stage3.adam_update`, `:248`; `project`, `:262`).

Projection rather than a sigmoid reparameterisation, for a concrete reason: **six of the fourteen genes sit exactly on a bound at the shipped genome** (`cy2` high, `t1`/`t2`/`t3` at `MIN_WALL_MM`, `R_rim` high). A squashing reparameterisation has vanishing gradient there, so the run would begin with six genes frozen. Projection has no such pathology—a gene on a bound whose gradient points inward moves immediately.

3 Glossary of repository-specific terminology

3.1 Structural and process vocabulary

Table 3: Structural and process vocabulary.

Term	Meaning	Where
§N / "section N"	A numbered section of `PLAN.md`, the project record. Cited constantly in comments and commit messages. §92 is the highest as of 2026-08-29.	`PLAN.md`
arc	An open unit of work with its own plan file (`FILLET_PLAN.md`, `HUBSHARE_PLAN.md`, ...). A "closed arc" is finished, its file deleted and its record folded into a `PLAN.md` section.	`PLAN.md:79-115`
M0...M9	Milestones of the original master plan. M2 = mesh validity, M3 = beam agreement, M4 = full-wheel linear FEA, M5 = geometric nonlinearity, M6 = real contact, M7 = the adjoint, M8a = the objective, M8b = the optimizer, M9 = buckling eigenvalue. Sub-milestones such as `M8b-i.6` are refinements.	`PLAN.md`, every study docstring

continued on the next page

Term	Meaning	Where
gate	A study driver that *exits nonzero on failure*. Nine of them make up `make studies`. Distinguished from a *calibration* (measures a constant) and a *machine description* (times, not physics).	`PLAN.md:1625` ff.
G1…G10, S1…S13, A1…A4	Individually numbered checks *inside* a gate. G-numbers are `study_gradient`'s, S-numbers `study_stage3`'s, A-numbers `study_beam_agreement`'s.	study docstrings
driver	A script in `studies/` that runs a measurement and writes `studies/study_x.json` (and `.jpg`). Committed *with* its artifacts, by house rule.	`PLAN.md` header
REDS	A closed arc: "the five inherited reds"—five long-standing failing tests, cleared in §31.	`PLAN.md:5095`
red	A failing test. `make test` reads `0 failed` since §31; `xfail_strict = true` means an `xfail` that *starts passing* is also a failure.	`pyproject.toml:31-46`
the shipped genome	`best_solution.json`, hash `09e8188`, promoted 2026-08-14. Distinct from the *GA/beam optimum* `36aed36`, which is pinned forever in `best_solution_ga_beam.json`.	`PLAN.md:257-263`
promotion	Moving a candidate into `best_solution.json`. Never a one-file change: the banner, drivers with measured constants, and `make svk` move too; `tests/test_promotion.py` carries the checklist.	`PLAN.md` header
env-opt / env-cad	The two virtualenvs.	`requirements-*.txt`
the hand-off	`wheel_fea.py` spawning `.venv-cad/bin/python src/wheel_step_export.py` as a subprocess at the end of a GA run. Deliberately non-fatal.	`wheel_fea.py` tail

3.2 Geometry vocabulary

Table 4: Geometry vocabulary.

Term	Meaning	Where
sector	One twelfth of the wheel, 30°. `SECTOR_DEG = 360/12`. The mesh builds sector 0 and rotates it eleven times.	`wheel_wheel.py:174`
block	One structured quadrilateral grid. Seven per sector unfilleted, eleven filleted, twelve for the rim tri-block.	`BLOCK_ORDER`, `:2394`
seam	A shared edge between two blocks, declared as `(block_a, side_a, block_b, side_b, dk, reverse)`. `dk` is the sector offset.	`_seam_table`, `:2420`
whole-edge seam	A seam where an entire block edge matches an entire other block edge. The alternative—a *partial-edge seam*—is avoided by splitting blocks, because whole-edge single ownership is "the whole safety net".	`:2404-2419`
span / thick / weld	The three element-count directions: `n_span` along the centerline, `n_thick` through the thickness, `n_weld` along the weld arc. Three counts are *not free* because they sit on shared seams.	`WheelConfig`, `:198`

continued on the next page

Term	Meaning	Where
smoke / coarse / medium / fine	Mesh refinement rungs. **Warning:** `wheel_mesh` and `wheel_wheel` both define these names and they are different meshes—a name passed between them once resolved silently against the wrong mesh and inflated a convergence order by 25%.	`CONFIGS`, `:252`; `wheel_mesh.py:123`
collar	The meshed annulus of the hub, `COLLAR_DEPTH_MM = 5.0` deep. Inside it the hub is treated as a rigid body, because a full disk cannot be one structured quad block (a polar grid degenerates at $r = 0$).	`:195`
rim band	The annulus from `rim_inner_radius()` (48.5) to `RIM_OUTER_RADIUS_MM` (50.0). 1.5 mm thick, and it holds about 32% of the wheel's compliance.	`:177`
junction / weld	Where a spoke meets a ring. Two per spoke, 24 in total.	`sector_blocks`, `:2209`
weld footprint	The arc, in degrees, that one junction occupies on its ring circle.	`weld_footprints_deg`, `:422`
void / slot	`SECTOR_DEG` minus the arc a spoke's material occupies at the hub—the empty arc between adjacent spoke roots, into which two fillets must grow from either side. Negative means adjacent spokes overlap.	`hub_void_deg`, `:467`
arrival angle	Angle between centerline and ring *tangent*: 0° tangential, 90° radial. See §2.5.	`arrival_angles`, `:380`
flank	One of the two offset curves bounding the spoke. $\eta = +1$ is "top", -1 is "bottom".	`offset_band`
flank orientation	$(\eta_{\text{hub}}, \eta_{\text{rim}})$, each ± 1: which flank straddles each ring. A **discrete topological decision**, not a smooth parameter. Only 16 of 60 feasible genomes share the shipped genome's $(+1, +1)$. Pinned for a whole Stage-3 run.	`flank_orientation`, `:559`
straddling flank	The one that crosses its ring circle. The other never crosses, which is what makes each junction four-sided.	`junction_stations`, `:594`
ring station	The arc-length fraction s at which a flank crosses a ring circle.	`ring_station`, `:296`
free arc fraction	$s_{\text{rim}} - s_{\text{hub}}$: the fraction of centerline that actually flexes (67–87%). **The wheel's dominant stiffness variable, and the beam model has no term for it.**	`:522`
P_t / P_c	The two re-entrant corners of an unfilleted junction. P_t is where the straddling flank crosses the ring—**the corner the real part has**, reproduced to 0.073 μm of arc. P_c is where the mesh's half end cap meets the circle—an artefact of the meshing, not a corner of the part.	`:1013-1030`
uncap	Replacing the junction's half end cap with the far flank's own continuation to the ring circle—the geometry the exporter's `_embed` actually builds. `UNCAP_DEFAULT = (True, 1.0)`; the hub and rim entries do *not* mean the same thing.	`:2115-2206`

continued on the next page

Term	Meaning	Where
embed	The exporter's straight tangent extensions burying each spoke end inside its ring, so the blunt end caps are absorbed by the boolean union. Its *argmax search* over 20001 candidates is the piece the mesh deliberately does not reproduce (it would be non-differentiable).	`wheel_step_export._embed`, `:248`
bite	Weld penetration in **root thicknesses**: overlap/$(t^2 \cdot \text{width})$. Derived rather than fitted: a spoke burying itself to depth d contributes $t\,d\,W$, so the ratio is exactly d/t. Replaced a raw mm^3 floor that was really a proxy for t and nothing else. Floor 0.25, honestly labelled a geometric floor, not a calibrated one.	`wheel_geometry.junction_bite`, `:395`
fold / fold margin	See §2.4.	`self_intersection_margin`, `:421`
fillet_a / fillet_b	The two blocks per junction of the eleven-block filleted sector. `a` is the boundary layer along the fillet arc; `b` is the wedge crossing the ring circle. They are split at N because one edge with two partners would be a partial-edge seam.	`:986-1010`
layer profile	The pair `(entry, end)` controlling the filleted boundary layer's width profile—a Hermite cubic. Since §85 the default is a *per-genome rule*, not a global pair.	`:1071-1140`
cliff	The value of `entry` at which a junction's layer reaches zero width. §88 found it has a **closed form**, $\max_u(Z - a(u))/b(u)$, agreeing with the 90-halving bisection it replaced to 2 ulp.	`_layer_cliff_from_scalars`, `:1537`
sector-fit clamp	Pulls a gene-derived fillet radius back inside the room its own sector has, at 95% of the limit. Took a drawn genome box from 10/16 building to 16/16. A clamp is *not* a gate: it keeps the genome and models a *smaller* fillet, which is honest only because `mesh.fillet_radii_mm` and `mesh.fillet_clamped` report what was actually built.	`SECTOR_FIT_CLAMP`, `:1171`
tri-block	The three-quad Y-partition of the rim junction at the faithful (`uncap` blend 0.0) rim, where the region becomes a curvilinear *triangle*. Built and measured (§53), not adopted.	`studies/study_tri_block.py`

3.3 Optimization and physics vocabulary

Table 5: Optimization and physics vocabulary.

Term	Meaning	Where
T1 / T2 / T3	The three cost tiers of the objective. **T1** is gene-space (microseconds, no mesh), **T2** mesh-space (milliseconds, `mesh_coords` only), **T3** solve-space (~ 0.7 s per phase). The tiering is economics, not organisation: a line search can re-evaluate T1 alone.	`wheel_objective.py:30-42`
barrier vs. objective term	See §2.6. `BARRIER_TERMS` / `OBJECTIVE_TERMS`, with an assert that every term is classified.	`:394-407`

continued on the next page

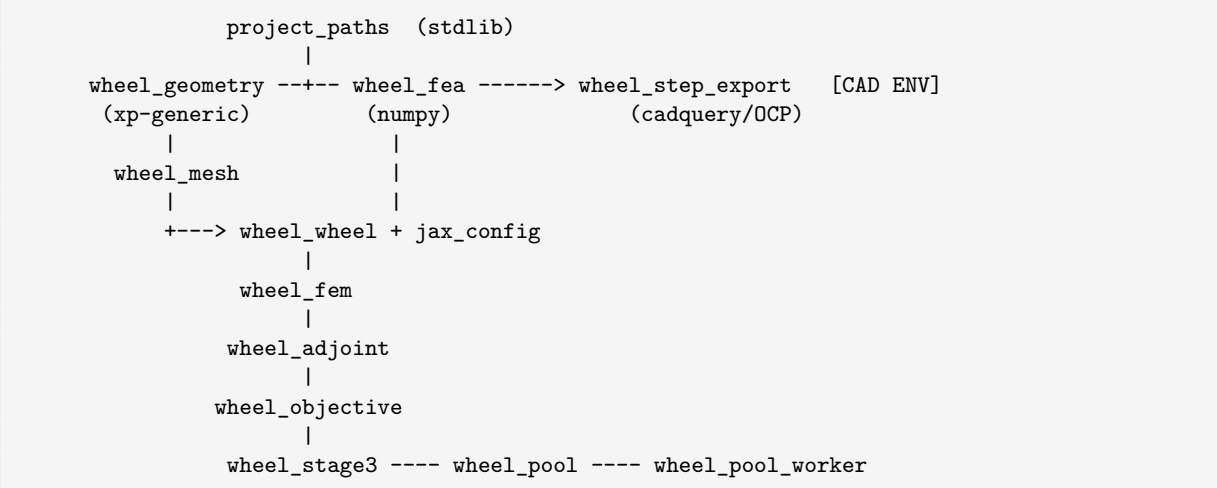
Term	Meaning	Where
QoI	Quantity of Interest—a scalar $Q(\texttt{coords}, \texttt{u\_full}, \texttt{y\_ground})$ the adjoint differentiates. The `QOI` registry holds `contact_force`, `strain_energy`, `mass`, `pnorm_stress`.	`wheel_adjoint.py:450`
axle drop	The wheel's deflection under load, in mm. Target 2.0. Measured as the prescribed ground indentation at which the contact force equals the service load.	`solve_wheel_contact`, `wheel_fem.py:1867`
phase / `phase_deg`	The rolling angle of the wheel under a stationary ground. Lives on `build_wheel(phase_deg=...)`, *not* on the load—the ground does not move. Putting phase in the load pushes the wheel sideways, a different load case, and silently breaks the 12-fold periodicity check.	`_sector_coords`, `:2583`
stencil	The set of phases one objective evaluation averages over. See §2.18.	`phase_stencil`, `:966`
utilisation (`util`)	$K_t\,\sigma_{\text{nominal}}$/allowable. 1.0 is the wall. Shipped genome: 0.8201 (hub), 0.5454 (rim).	`t3_terms`, `:1045`
the knee	`MARGIN_KNEE_UTIL = 0.80`. `stress_margin` is soft_barrier(`util` − 0.80)—the same function as the `stress` wall with the knee moved in. Encodes the policy "margin below 0.8 is close to worthless, above 0.95 close to priceless". Explicitly labelled a judgement, not a measurement.	`:343`
`min_sj`	The mesh-validity barrier, summed over *every* element rather than applied to the worst—because `min` is a max-like kink whose gradient is a delta on whichever element is currently worst, and because summing also tells the optimizer *how many* elements are marginal. Target 0.2.	`t2_vector`, `:931`
coupling term	The $(\partial Q/\partial\delta)(d\delta^*/dp)$ half of the total gradient at constant force. See §2.16.	`service_qoi_value_and_grad`
frozen / freezing	Holding a discrete decision fixed so a derivative exists: flank orientation, seam ownership, fillet roots, the clamp's decision. Always *named*, never hidden.	`mesh_coords` docstring
inert term	A term with a large loss value and a tiny gradient—dominates the table, contributes nothing to descent. `400*n_inflections` was this in pure form. Gate 8 exists to detect it.	`studies/study_objective.py`
event	A structured record in a run log of something the optimizer decided: `t1_reject`, `orientation_flip`, `fidelity_check_failed`, abandonment.	`descend`, `:410`
selection key	The three-tier promotion sort. See §6.7.	`:195`
`make <target>`	Every measurement in this tree has a Makefile target: `make fillet`, `filletblock`, `corner`, `hubcap`, `gci`, `triblock`, `svk`, `studies`, `test`.	`Makefile`

4 Architecture and module breakdown

```
src/
  project_paths.py     35 lines   ROOT/SRC/STUDIES/EXPORT.  STDLIB ONLY -- imported by
```

```
                                wheel_fea, which the CAD env imports.
  jax_config.py        36       `jax.config.update("jax_enable_x64", True)`, and raises
                                loudly if something traced first.  float64 is mandatory:
                                at float32 a Newton solve cannot reach 1e-10, an FD check
                                has no plateau, and the patch test's 1e-12 is below eps.
  wheel_genome.py     183       names, bounds, normalisation, genome_hash, record I/O.
                                numpy+stdlib only.
  wheel_geometry.py   436       THE GEOMETRY KERNEL.  Bezier, taper, offset band,
                                curvature, fold margin, junction bite.  Dual-backend via
                                `xp=`; never imports jax.
  wheel_mesh.py       391       One spoke block: sampler, grid connectivity (Q4/Q9),
                                boundary node sets, quality metrics (scaled Jacobian,
                                aspect ratio, areas).
  wheel_fea.py       1713       STAGE 1/2.  Constants, GENE_SPACE, Castigliano beam model,
                                the nine loss terms, the pygad driver, the fillet-arc
                                plotting helpers, and the CAD hand-off.  numpy only.
  wheel_wheel.py     3709       THE FULL 360 DEGREE MESH.  Ring geometry, block builders,
                                Coons patches, the filleted eleven-block sector, seam
                                tables, union-find assembly, WheelMesh, the differentiable
                                `mesh_coords` + jitted `coord_fn`, area and quality
                                reports.  Largest and most intricate file in the tree.
  wheel_fem.py       1914       THE FE KERNEL.  Shape functions, energy, autodiff'd force
                                and tangent, DofMap, penalty contact, linear and Newton
                                solvers, stress recovery, spoke/wheel/contact problems.
  wheel_adjoint.py    759       THE GRADIENT.  QoIs, the adjoint solve, load-controlled
                                total derivatives, LOBPCG buckling eigenpair.
  wheel_objective.py 1461       THE STAGE-3 SCALAR.  Fourteen terms in three tiers, the Kt
                                stress constraint, the hub fillet cap model, phase
                                stencils, jit caches, loss breakdown reporting.
  wheel_stage3.py    1093       THE DRIVER.  Projected Adam (and an L-BFGS-B alternative),
                                step-reject policy, selection key, run records, CLI.
  wheel_pool.py       408       Parent half of the phase worker pool.  stdlib+numpy, NO jax.
  wheel_pool_worker.py 103      Child.  Pins threads BEFORE its first import.
  wheel_step_export.py 1248     THE EXPORTER (CadQuery env).  Profile construction,
                                embedding, boolean union, fillet ladder, health checks,
                                STEP write, manifest.
studies/              ~19 drivers + their committed .json/.jpg artifacts
tests/                25 files, 503 test functions
```

4.1 The import graph, and the constraint that shapes it



The horizontal line matters more than the vertical one. Everything to the left of and including `wheel_fea` must import **with numpy alone**, because `wheel_step_export.py` runs in the CadQuery

interpreter and imports `wheel_fea` and `wheel_geometry` from it. A single top-level `import jax` anywhere in that half breaks the exporter, and `tests/test_import_hygiene.py` runs `import wheel_fea` in a jax-free subprocess to prove it has not happened.

This is also why `wheel_wheel.mesh_coords` imports `jax.numpy` *lazily*, inside the function, and why `wheel_geometry` takes `xp` as a parameter instead of importing an array module. Note also that `MAX_ARRIVAL_DEG`, `MIN_FOLD_MARGIN_MM` and `MIN_JUNCTION_BITE` live in `wheel_geometry` rather than where they are used, because their *producer* and *consumer* are on opposite sides of the interpreter boundary and this is the only module both can import.

`src/` reaches the interpreter three ways—`pyproject.toml`'s `pythonpath` for pytest, `export PYTHONPATH` in the Makefile, and the root `conftest.py` (which also seeds `os.environ` so the three subprocess-spawning tests behave identically under bare `pytest` and under `make test`). A package with `__init__.py` was rejected precisely because an `__init__` importing jax-dependent siblings would break the CAD environment.

4.2 Caches, and why they are keyed the way they are

There are three JIT caches in this tree, all following the same idiom and all with the same failure mode documented.

`wheel_wheel._COORD_FN_CACHE` (`:2760`), the jitted genes → coords map. The JIT is load-bearing, not tuning: `sector_blocks` unrolls a fixed-count Newton over `n_curve` stations for every ring crossing, so tracing is expensive, and `jax.vjp` on an *untraced* closure re-traces on **every call**—measured at 0.774 s against 0.05 s for the entire rest of the adjoint. Without the cache, 97% of a Stage-3 gradient is re-deriving a jaxpr that never changes.

The cache **cannot be keyed on the mesh object**—the obvious thing, and wrong. Every finite difference, every sweep point and every optimizer step builds a *new* mesh at new genes, so a per-object cache misses on literally every call it exists to serve while looking like it works. The key is the *static recipe* instead: element counts, orientation, ownership bytes, phase, `repr(uncap)`, and the fillet recipe. Genes are the traced *argument* and must never be in the key. Correspondingly, the filleted mesh's frozen roots are passed as a *traced array argument* rather than closed over—because they depend on the genome, and closing over them would put a design into the jaxpr and re-trace on every step (2.78 s to trace once, 0.0095 s per call after; 2.78 s *every* call if closed over).

`repr(uncap)` in the key is a correctness entry, not a performance one. Its absence was invisible while `UNCAP_DEFAULT` was `False`; when §38 flipped the default, a mesh built with an explicit `uncap=False` got traced coordinates from the *other* geometry—**0.448 mm of error** against the 10^{-9} mm the path is documented to hold to.

`wheel_objective._T1_CACHE` (`:872`) and `_KT_CACHE` (`:521`) follow the same pattern for the T1 term vector and the K_t value-and-gradient. T1's jit took a full evaluation's `t1_vector` from about 1.06 s to 0.0054 s—a roughly 195× improvement that mattered disproportionately, because once the phase loop was parallel, T1 and T2 (which run in the parent on every path) became the whole of Amdahl's serial fraction.

4.3 The phase pool

`wheel_pool.py` runs the eight per-phase solves of one evaluation in eight processes. It is deliberately *not* `multiprocessing.Pool`, for three load-bearing reasons.

1. **Thread settings must be in place before numpy and JAX are imported.** `OMP_NUM_THREADS` and its siblings are read by OpenBLAS/MKL at import and `XLA_FLAGS` by XLA at its first trace; `spawn`'s `initializer=` fires *after* the unpickling that already imported everything. `fork` would be early enough and is unusable on two counts (forking an initialised

JAX process is a documented hazard; `fork` does not exist on Windows). So the pool uses an explicit `Popen(env=...)`.

2. **Phase slots must pin to workers**, so slot i always goes to worker i mod n_{workers} and each worker only ever traces its own phases' jaxprs.

3. **Results must combine in slot order.** Floating-point addition is not associative, so an as-completed reduction would make the aggregate depend on the scheduler. In slot order the arithmetic is identical, which is what lets S13 gate *pooled = serial exactly* rather than to a tolerance.

`XLA_FLAGS` deserves its own note, because it is a **correctness** setting here. Measured: two plain serial runs of one `coarse` adjoint, in two separate interpreters with no pool anywhere, agree on every forward value to the bit and disagree on the *gradient* by 3.33×10^{-16}—XLA's CPU intra-op thread pool is sized from the machine and its parallel reductions do not associate the same way twice. Pinned to one thread, the same comparison is exactly zero, and it costs 19.84 s against 20.43 s. Nothing in OMP/MKL/OPENBLAS/NUMEXPR touches that pool.

Nothing unpicklable crosses the boundary: `WheelMesh.__slots__` holds `_coord_fn`, a jitted `PjitFunction` after the first `mesh_coords` call. So the worker builds its own mesh from the genes and returns only the leaves `t3_terms` reads—three floats and three 14-vectors. The reply is shaped *exactly like* the serial adjoint dictionary, so the loop body in `t3_terms` is literally the same lines on both paths.

Measured: 3.95× at 8 workers on 16 cores; a production run projects 46.46 h down to 11.77 h.

5 Full workflow walkthrough

Let me trace one concrete path end to end: **Stage 3 takes one Adam step** on the shipped genome. This is the deepest path in the system and touches almost everything.

Step 0 — the CLI

```
.venv-opt/bin/python src/wheel_stage3.py --start best --steps 100 \
    --config medium --kinematics svk --min-wall 1.2 --workers -1
```

`main()` (`wheel_stage3.py:932`) loads `best_solution.json` via `load_genes` (`:844`), converts the 14-key dictionary to an ordered vector with `wheel_genome.genes_to_vector`, and normalises it into the unit box. `-min-wall 1.2` calls `wheel_fea.set_min_wall`, which rebuilds `GENE_SPACE`'s thickness bounds—this is *not* cosmetic: `descend` projects its start into the box before it steps, so loading the shipped 1.2 mm genome without this flag would silently lift all four thicknesses to 2.0 and optimise a different, heavier wheel without a word of complaint (`wheel_fea.py:216-236`).

Step 1 — pinning the topology

`descend` (`:410`) computes the **flank orientation** once, from the starting genes:

```
orientation = WW.flank_orientation(wg.denormalize(z, low, high), wcfg, span_mm=span_mm)
```

`flank_orientation` (`wheel_wheel.py:559`) samples the two flanks at the endpoints and asks which one is inside its ring circle. The answer is a pair of ± 1. It is **pinned for the whole run** and threaded through every `build_wheel` call thereafter, because it is a discrete topological decision: a step that flips it changes which block owns which node, and the gradient of that step does not exist. It is *also* re-derived every step for reporting, and a disagreement is logged as

an `orientation_flip` event—the run keeps going on the pinned topology, and the event says the trajectory has walked somewhere the frozen answer is stale.

Then the pool is created (`WP.PhasePool(n_workers)`), owned by `descend` rather than by the `Evaluator`, so the `finally` that closes it sits next to the loop a Ctrl-C can interrupt.

Step 2 — drawing the phase stencil

```
phases = WO.phase_stencil(n_phase=8, n_sub=8, scheme="rqmc", rng=rng)
```

Eight angles in $[0°, 30°)$, on the fixed 64-point lattice (§2.18).

Step 3 — building eight meshes

`Evaluator.__call__` (`:377`) calls `WO.phase_meshes(genes, cfg, phases, orientation=orientation)`, which is twelve-fold `build_wheel(genes, cfg, phase_deg=p, orientation=orientation)`. (Pooled, the parent builds only mesh 0—T2 reads `meshes[0]`—and each worker builds its own.)

Inside `build_wheel` (`wheel_wheel.py:2869`):

1. `_sector_coords` (`:2583`) calls `sector_blocks` (`:2209`), which builds sector 0's seven node grids.
   - `global_sampler` (`:275`) wraps `wheel_mesh.band_sampler` and shifts it into the global frame, so *every* block that touches the band goes through the same closure and shared nodes agree bitwise rather than to $O(h^2)$.
   - `junction_stations` then `ring_station` (§2.17) finds s_{hub} and s_{rim} to 10^{-15}.
   - the **spoke** block is `sample(s_grid[:,None], eta_grid[None,:])` — the offset band of Equation (5), sampled uniformly in arc length $\times$ η.
   - each **junction** is a Coons patch whose four sides are: the spoke's own end cross-section (the same array, so the seam is exact by construction), an arc along the ring circle, the far flank sampled at $-\eta$, and a straight segment to the far end. Corner angles come out 79.5/90/90/79.5 degrees rather than the 10.5 the near-tangent arrival suggests—that is the §2.5 inversion paying off.
   - each **ring** is split into a `_weld` polar block (spanning exactly the weld arc) and a `_free` polar block (the rest of the sector), both laid out in increasing θ so that `weld.i1` $\rightarrow$ `free.i0` $\rightarrow$ next `weld.i0` holds for every genome and the tiling closes.
2. The seven grids are rotated into twelve sectors by `_rotate` and concatenated. **Phase is applied here**, not to the load.
3. The **seam table** is walked; a `_UnionFind` merges declared-equal nodes; the seam error is measured *before* the non-owners' coordinates are discarded.
4. Nodes are relabelled to a compact global numbering; `owners` (the raw index of each merged node) is saved on the mesh—this is what `mesh_coords` will index with.
5. Connectivity is emitted per block by `wheel_mesh.grid_connectivity` and passed through `_orient_elements` to flip left-handed polar blocks.
6. `_node_sets` / `_edge_sets` build `hub_tie`, `rim_outer` and friends. Edge *segments*, not just node sets, because a distributed load needs consistent nodal forces and the correct weights on a quadratic edge are $1/6, 4/6, 1/6$—not equal thirds.

Out comes a `WheelMesh` carrying coordinates, connectivity, config, per-element block and region tags, node and edge sets, `seam_error_mm`, `owners`, `orientation`, `phase_deg`, `uncap`, and (if filleted) the applied radii and the frozen root record. At `medium` this is about 104k degrees of freedom.

Step 4 — the objective

`WO.objective(z, cfg, normalized=True, ...)` (`wheel_objective.py:1308`) denormalises, then runs three tiers.

T1 (`t1_vector`, `:774`)—genes only, no mesh, microseconds via the jitted cache. Seven terms, returned as a *vector* so `jax.jacrev` gives every term's gradient in one backward pass (which is what the per-term inertness census needs):

- `x_order`—control points must stay ordered with a 2 mm gap, plus a fold-back barrier on $-\min(\mathrm{diff}(x))$;
- `hub_overlap`—root thickness plus two fillet radii plus clearances must fit the chord $2R_{\mathrm{hub}}\sin(\pi/12) \approx 6.57$ mm available to one sector;
- `smoothness`—**rewritten** for differentiability. The old `400*n_inflections` was an integer from `count_nonzero` behind a data-dependent mask with *exactly zero gradient* while being 20.6% of the GA's loss. It is replaced by a curvature-rate integral $\int (d\kappa/ds)^2\, ds$ plus a smooth sign-reversal product $\sum \max\left(0, -\kappa_i\kappa_{i+1}/\kappa_{\mathrm{ref}}^2\right) \Delta s$;
- `fold`—§2.4;
- `arrival`—§2.5;
- `fillet`—a closed-form geometric feasibility margin for each of four fillet corners, barriered individually so one infeasible fillet cannot be averaged away by three comfortable ones. **This is what gives `R_hub/R_rim` a live gradient** even though the default mesh models no fillets;
- `fillet_cap`—`R_hub` may not exceed `hub_fillet_cap_mm`, the largest hub fillet the part can actually build (§6.5). A separate term from `fillet`, deliberately: `fillet` asks whether a circle fits in one re-entrant *corner*, this asks whether it fits in the *slot* two spokes leave between them. Different mechanism, different gradient, and every census in the tree reads at term granularity—folding them together would make "is the cap binding?" unanswerable from the loss table.

Plus `buckling`, computed from the numpy beam surrogate with an explicitly asserted *zero gradient*—currently exactly 0.0 (ratio 0.139 against a threshold of 1.0), and the gate asserts it *stays* zero, so the day it starts to bite the census fails and says so.

T2 (`t2_vector`, `:931`)—mesh-space, via `mesh_coords` alone:

```
coords = WW.mesh_coords(genes, mesh)          # differentiable, jitted
mass_g = sum(element areas) * width * density
min_sj = sum_e soft_barrier(0.2 - sj_e)
```

T3 (`t3_terms`, `:1045`)—one `service_qoi_value_and_grad` per phase (§2.16), each of which is a secant to service force, a re-solve at the found indentation, one stiffness assembly, one factorisation, and back-solves for `contact_force`, `pnorm_stress` and whatever else was requested, all sharing one mesh vjp. Then:

- `deflection` $= 2500\left(\frac{\overline{\mathrm{drop}}-2.0}{2.0}\right)^2$, the mean over the stencil;
- `stress`—a smoothed maximum over the *phase* samples (p-norm at $p = 8$, replacing CVaR, which at 8 samples is "the worst one or two" and whose gradient jumps as the set switches) fed into the $K_t\,\sigma_{\mathrm{nom}} \le$ allowable barriers, one per junction (§2.14);

- `stress_margin`—the same two utilisations, priced instead of walled, with the knee at 0.80;
- `phase_ripple`—std/mean of the drops over the stencil.

Note that the two exponents in T3 are **not the same quantity**: `stress_phase_p = 8.0` aggregates the eight phase samples; `stress_gauss_p = 4.0` is the volume-weighted p-norm exponent over the Gauss points inside one solve.

`objective` sums the fourteen values, sums the fourteen gradients, applies the unit-box chain rule if `normalized`, and returns `(value, grad, breakdown)` where the breakdown carries per-term value, share, gradient norm and gradient share.

Step 5 — the step

Back in `descend`:

```
lr_t             = cosine_lr(lr, i-1, steps)
g_clipped, g_norm = clip_global_norm(grad, 1.0)
delta, m, v      = adam_update(g_clipped, m, v, i, lr_t)
z_trial          = project(z - scale*delta)
```

Then the **T1 pre-check**: `t1_barrier_sum(z_trial, ...)`—microseconds, no mesh, no solve—and if the trial's barrier sum exceeds $\max(\texttt{T1\_REJECT}, b_{\text{here}})$ it is refused *before any mesh is built*.

`T1_REJECT = 1.0e4`, and the number is the whole lesson (`wheel_stage3.py:143-167`). It was 1.0, with the rule "never step further into the wall". That **deadlocked** a probe: the shipped design carries `hub_overlap` = 52.13, the stress-reducing direction raises it to 55.41, and halving only walks it back to 52.54, so every trial at every scale was refused—194 events, zero accepted steps, thirty-five minutes standing still. The error was conceptual: every barrier is *already* a term in the objective with a weight chosen to price it, so a pre-check that also forbids increasing it duplicates and overrides that job—and what it overrides is exactly the constrained trade the run exists to make. The screen now keeps only the job the objective cannot do: refusing to spend a solve on geometry that will not mesh. Healthy barriers are tens; an analytically infeasible fillet scored 112,000.

If the objective *raises*—`NewtonDivergedError`, or the $dF/d\delta \leq 0$ guard—the step is halved and retried up to `-max-rejects`. On exhaustion the iterate is restored and the learning rate decays. `wheel_objective.py` contains **no `try/except` anywhere**, deliberately: the caller owns the policy, and the policy is in `wheel_stage3`. This matters beyond robustness, because slope ≥ 0 in the Newton loop is the tangent losing positive definiteness, and swallowing it as a large loss would let the optimizer walk into a buckled design and score it.

If accepted, the **warm vector** for the next step is the per-phase indentations, read straight off `breakdown["report"]["rows"]`—because `service_qoi_value_and_grad`'s `delta0` *is* the number reported as `axle_drop_mm`. Between two Adam steps the design barely moves, so the previous answer is a far better guess than a cold linear solve.

Every N steps, an optional **fidelity check** forward-evaluates the just-accepted iterate at a second config (`medium`), T3 tier only, and *discards the gradient*—proven by a test that runs the same seed with the feature on and off and asserts `z`, `grad` and `loss` are bit-identical at every step. Pure observation: it can report on the trajectory, never redirect it.

Step 6 — selection and persistence

Every step is scored by `selection_key` (§6.7) and the best is tracked. The run record (`_record`, `:665`) is written incrementally, so a killed run leaves a readable partial. At the end, `-best-out` writes a genome record through `wheel_genome.save_record`—genes plus top-level `search`, `loss_terms` and `metrics` blocks.

Step 7 — promotion and export

A promotion moves the candidate to `best_solution.json`, and by house rule that is **one atomic commit that is never one file**: the banner, the drivers carrying measured constants, and `make svk` move with it, and `tests/test_promotion.py` carries the checklist.

Then `make export` runs `wheel_step_export.py` in the CAD environment:

1. `load_genome` reads `best_solution.json`; `warn_if_stale` compares mtimes.
2. `spoke_edges_global` re-derives the two flanks from `wheel_fea.thicken_3taper_curve`—the *same* geometry kernel the mesh uses.
3. `_embed` (`:248`) appends **straight tangent segments** to each flank end, burying the spoke root 0.5 mm inside the hub disk and running the tip 0.25 mm past ⌀100. Straight, because the previous approach—moving the last point and interpolating a spline through it—put a *self-intersecting loop* in every spoke: the first span jumped 3 mm while its clamped tangent pointed 40.7° against a local curve direction of 76.1°. The union trimmed the loops away but left a 0.041–0.067 mm near-cusp running the full 22.4 mm as a knife edge. OCC calls that valid (`BRepCheck_Analyzer` does not test for cusps) and Parasolid refuses to build a face across it, drops the face, and **the spoke imports open**. A straight line adds no curvature, cannot cusp, and continues the flank's own end tangent so the join is G^1.
4. `build_profile` unions hub disk + 12 spokes + rim band into one planar face, extrudes it, and clips to ⌀100.
5. `_junction_edges` finds the full-height vertical edges and measures each one's *material wedge angle* by classifying sample points against the solid. A corner is worth filleting when its wedge exceeds 180° (re-entrant). The five families separate with a wide margin: 354° (the spoke-to-spoke notch at the hub, fillet), 152° (convex, skip), 180° (straight through, no corner), 194–356° (the rim junctions, fillet), 178° (the OD trim seam, skip).
6. `fillet_junctions` walks a *ladder*: request the genome's radius; if OCC refuses, retry at 0.85× (`FILLET_LADDER_DECAY`), down to `MIN_CURVATURE_RADIUS_MM = 0.25`. All 48 corners (24 hub + 24 rim) are filleted on the shipped genome, at the requested radii, with `kt_error` 0.0% at both junctions.
7. `step_health` runs a surface census, a minimum-curvature scan (a fault detector: 0.25 mm is well under any fillet requested, so a violation always means a construction fault), and `despecialize` (converting swept and offset surfaces to canonical ones, which is what some importers need).
8. `_export_with_settings` writes AP214 STEP; `report` writes `export/wheel_step_manifest.json` with the genome hash, junction overlaps *and bites*, the fillet radii OCC actually managed versus what the optimizer priced, and the solid volume with and without fillets.

For the shipped genome: 39224.5 mm^3 solid, 36145.8 mm^3 unfilleted, **3078.77 mm^3 of fillet = 7.85% of the solid**, 48.64 g of PLA.

6 Key algorithms, in detail

6.1 `mesh_coords` — the differentiable mesh

`wheel_wheel.mesh_coords(genes, mesh)` (`:2633`) returns the mesh's node coordinates as a differentiable function of the genes. The trick is what it *freezes*:

- the **flank orientation**—taken from `mesh`, not recomputed;
- the **seam ownership**—`coords_all[mesh.owners]`;

- on a filleted mesh, the tangency and ring-crossing **roots**, seeded from the eager build and refined by one Newton step;
- the four geometric refusals `_fillet_curves` can make.

So this is **the derivative of a fixed mesh whose nodes move**, which the docstring argues is the only derivative that means anything: a step that flips the orientation has no finite-difference plateau, and *should not*—it is a real discontinuity of the design space, not a defect in the gradient.

Crucially, `build_wheel` itself is deliberately *not* made traceable. Its seam check, element orientation pass and validity guards all need concrete coordinates, and they are the reason the mesh can be trusted; wrapping them in a "skip when tracing" branch is exactly how guards stop running. Instead the traced half is factored out (`_sector_coords`) and shared, and `mesh_coords` *reproduces* the eager result—checked to 10^{-9} mm by `study_gradient.py` gate 3 and `tests/test_gradient.py`.

6.2 The p-norm family, and why `max` never appears in a gradient

Three separate places replace a `max` or `min` with a smooth aggregate, and each records the same argument in its own terms.

Quantity	Naive	Used instead	Why
peak stress over Gauss points	`max`	volume-weighted p-norm, $p = 4$	argmax jumps as the contact patch sweeps; also `max` diverges under refinement
peak over phase samples	CVaR	p-norm, $p = 8$	at 8 samples CVaR is "the worst one or two" and its gradient jumps as the set switches
worst element quality	`min sj`	$\sum_e \mathrm{soft\_barrier}(0.2 - sj_e)$	the gradient of `min` is a delta on whichever element is worst; and the sum tells you *how many* are marginal
effective fillet radius	$\min(R, \mathrm{cap})$	`smooth_min(R, cap, k)`	needs C^1 at the tie, but **exact** outside a blend of width k

Table 6: Four places a hard extremum was replaced, and the reason in each case.

`smooth_min` (`wheel_objective.py:424`) is worth a close read. The blend width is $k = (1 - 0.85) \cdot \max(\mathrm{cap}, 0.25)$, i.e. **one rung of the exporter's fillet ladder** (the `max` only keeps k positive when adjacent spokes have closed the slot entirely)—narrower would model a distinction the exporter cannot make, wider would bias the effective radius below a cap the exporter builds *at*. Exactness outside the blend is the property it exists for, and it is a claim a test asserts. Two obvious alternatives fail it: the hyperbolic smooth-min $\frac{1}{2}\big(a + b - \sqrt{(a-b)^2 + \epsilon^2}\big)$ is C^∞ but never exact (measured 1.3% below the cap at the shipped genome—inventing a fillet reduction nothing measured), and log-sum-exp is scale-coupled and needs overflow guarding.

The docstring also records a subtle autodiff bug *at exactly* $a = b$. Two primitives are non-differentiable at the tie and autodiff picks a subgradient for each: `jnp.abs` returns 0 at 0 (killing the blend term) and `jnp.minimum` hands the full 1.0 to its first argument. The sum was 1.0 against a true two-sided derivative of 0.5—the function is perfectly smooth *through* the tie, so this was an artifact of the spelling, not of the mathematics. Writing the minimum symmetrically as $(a + b - |a - b|)/2$ fixes it, confined to the blend by a `where` because the symmetric form is not bit-exact. Measure-zero and never observed to bite—and worth fixing because the margin term drives `R_hub` toward its cap on purpose, which makes the tie an *attractor* rather than an accident.

6.3 `adjoint_grads` — one factorisation, N solves

```
K   = assemble_stiffness(coords, conn, u_full, ...) + contact.stiffness(coords, u_full)
Kr  = (T.T @ K @ T).tocsc()
back_solve = spla.factorized(Kr)                              # ONE LU
_, vjp = jax.vjp(lambda v: WW.mesh_coords(v, mesh), genes)    # ONE trace

for each QoI Q:
    value  = Q(c, u, y)
    dQ_dc, dQ_du, dQ_dy = jax.grad(Q, argnums=(0,1,2))(c, u, y)
    adj_r  = back_solve(-(T.T @ dQ_du))                       # ONE back-substitution
    adj_f  = T @ adj_r
    d_dc, d_dy = mixed(c, u, y, adj_f, *static)               # grad_c and d/dy of adj . grad_u Pi
    grad   = vjp(dQ_dc + d_dc)[0]                             # chain to the 14 genes
```

What is shared is what is expensive: the reduced tangent K_r is identical for every quantity evaluated at the same converged state—only the right-hand side changes—and the mesh vjp is the bigger saving at 0.774 s versus 0.05 s for everything else. Since the load-control coupling needs both `pnorm_stress` and `contact_force` at the same state, this is the difference between one gradient's cost and two.

There is also a guard on the adjoint solve itself: a non-finite multiplier means the tangent at the converged state is singular, "which means the forward solve stopped at a state Newton could not have converged to."

6.4 The three searches, and the order to attack them in

`wheel_adjoint.py`'s docstring generalises a lesson the fillet arc paid for three times, and it is the single most transferable idea in the repository. Three bracketed searches appeared inside the geometry, each blocking a gradient. Each took a *different* instrument.

1. **`ring_station`'s crossing solve**—replaced by a *frozen root plus one Newton step*. Unrolled Newton's derivative converges to the implicit-function derivative, so the derivative is right without any wrapper.
2. **`_fillet_tangency`'s tangency solve**—the same treatment. A `custom_vjp` around the bisection would have been the *wrong* instrument, "because there is nothing wrong with the bisection's derivative that a wrapper could fix. There is no derivative in a `sign` comparison to wrap."
3. **The layer cliff**—90 halvings of a bracket, and the frozen-root step does not apply, because the answer is not a root of an equation the traced path re-evaluates but the *output* of ninety halvings. §85 refused the gradient outright. What removed the refusal was neither a wrapper nor a frozen root: **the cliff had a closed form and nobody had looked for one.**

The layer width profile is a Hermite cubic that is *affine in* `entry`:

$$H(u) \;=\; h_{00}(u)\,w \;+\; h_{01}(u)\,(e\,w) \;+\; h_{10}(u)\,(\mathrm{entry}\cdot k) \;=\; a(u) \;+\; \mathrm{entry}\cdot b(u), \tag{42}$$

writing w for `wall`, e for `end` and k for `layer_k`, with $b(u) = u(1-u)^2 k > 0$ strictly inside $(0,1)$. So $H(u) \le Z$ exactly when $\mathrm{entry} \le (Z - a(u))/b(u)$, and the sampled minimum reaches zero exactly when one of them does:

$$\mathrm{cliff} \;=\; \max_{u}\; \frac{\texttt{LAYER\_CLIFF\_ZERO} - a(u)}{b(u)}. \tag{43}$$

Exact where the bisection was approximate (the two agree to **2 ulp**, worst relative 4×10^{-16}, over 98 junction pairs and 48 genomes), and differentiable where the bisection had ninety `sign` comparisons. It also costs arithmetic instead of thirty filleted sector builds, which is what makes a per-genome layer profile adoptable at all.

The order to try things in is therefore: is the search unnecessary; is its answer a root worth freezing; and only then is it a discrete decision to be classified. Two of this project's three searches ended at the second question and one at the first; none needed a `custom_vjp`.

6.5 `hub_fillet_cap_mm` — a model built from a falsification

The largest hub fillet the part can actually build is the **minimum of two limits** (`wheel_objective.py:165-300, :704`), and the structure of that `min` is the result of a measurement campaign (`make hubcap`) that overturned the original single-term model.

- **The slot limit.** Two fillets grow into the gap adjacent spoke roots leave on the hub circle, so each may claim a share of `hub_void_deg`.
- **The root-thickness limit.** A fillet in the re-entrant corner where a spoke of thickness t_0 meets the hub circle cannot grow past roughly half that thickness.

The 24 hub corners are **two families of twelve**, named by wedge angle rather than by rank: a *square-on* family at 266–270° (the thickness mechanism) and a *near-cusp* family at 294–332° (the slot mechanism). Which one binds is *observed*, not assumed—and it is not the same family on every design. At $t_0 = 1.2$ (where everything ships) the square-on family binds; on the $t_0 = 2.55$ Stage-2 elites the near-cusp family binds.

That last fact is why the original calibration was wrong in a way its own error bars could not show: `HUB_CAP_THICKNESS_SHARE = 0.52` had been fitted to five ratios inside a 0.9% band—**all from designs where that family does not bind**. The constant was calibrated tightly against a family that was never the limit. The result was a cap that over-promised on all five designs by 1.07× to 1.62×, while the driver's own gate read green because it compared against the *larger* of the two families.

The re-fit added an arrival dependence:

$$\begin{aligned}\frac{\text{square-on threshold}}{t_0} &= S_{\text{thick}} \;-\; S_{\text{arr}}\,(1-\cos\theta_{\text{hub}}) \\ &= 0.505 \;-\; 0.48\,(1-\cos\theta_{\text{hub}}),\end{aligned} \tag{44}$$

writing S_{thick} for `HUB_CAP_THICKNESS_SHARE` and S_{arr} for `HUB_CAP_ARRIVAL_SLOPE`. The $(1-\cos)$ form was chosen over a marginally better quadratic **because it stays positive across the whole physically reachable** $[0°, 90°]$—a quadratic crosses zero at 87.4° and would assert that a near-radial spoke admits no fillet at all, which is a claim nothing measured. Out of sample on two designs in neither fit, the law is under by 3.1–3.3%, the same margin it carries in sample. And the two families run in *opposite* directions in arrival angle, which is why the buildable radius is a *peaked* function of it and why five designs disagreed about the sign of the effect until a controlled sweep separated them.

The re-fit brings the five caps from 1.067/1.615/1.199/1.463/1.479 of the worst measured corner down to 0.979/0.969/0.827/0.967/0.969. Note also that `fillet_cap` carries **no safety factor**, deliberately, and the comment says why: a factor stacked on top would be a threshold with no measurement behind it. The conservatism lives in the fit, where it is measurable.

6.6 `t3_terms` and the two-junction stress constraint

Already covered mathematically in §2.14; two implementation notes remain.

The `(name, factory)` form is load-bearing. To reach `_qoi_pnorm_stress` at $p = 4$ rather than the module default of 30, T3 passes `("pnorm_stress", lambda prob: WA._qoi_pnorm_stress(prob, p=stress_gauss_p))`. The bare string would take the module default and no argument could change it. The *name* stays `"pnorm_stress"` so every downstream key is unchanged.

Probes cost nothing. The p-norm is a pure function of the converged displacement field, so any number of exponents can be read off the field the adjoint already ran on. Each probe is pushed through the *same* `_stress_aggregate` the constraint uses, so `pnorm_by_p[stress_gauss_p]` reproduces the report's own `stress_utilisation` exactly—the probe is checkable against the thing it probes. **No gradient is taken at a probe** p: a convergence study reads values, and one adjoint per exponent per phase would turn a 14-minute ladder into an hour for numbers nothing reads.

6.7 `selection_key` — which iterate of a run is promotable

```
violation = sum over BARRIER_TERMS of term value
if violation > 0:              return (2, violation, loss)
slack = hub_fillet_cap_mm - R_hub
if slack < MIN_CAP_SLACK_MM:   return (1, -slack, loss)
                               return (0, 0.0, loss)
```

Three tiers, most shippable first, sorted lexicographically. Tier 2 always ranks last **but is still ranked**, so a run in which nothing is feasible reports its least-violating step rather than nothing at all.

`MIN_CAP_SLACK_MM = 1e-3` is not zero, and the reason is worth internalising: **feasibility is fidelity-dependent.** On 2026-08-11, step 82 of a re-descent cleared the cap at `medium` by 53 nm and violated it at `coarse` by 93 nm. A barrier reading exactly 0.0 therefore certifies nothing on its own; it certifies that *this mesh* saw no violation. 10^{-3} mm is four orders above that crossover and four below what the step actually promoted was holding.

Ranking tier 1 by *slack* before *loss* is also deliberate: inside a 1 μm band the loss differences are numerical noise against a feasibility question the mesh cannot resolve, so the tiebreak that means something is distance from the knee.

6.8 The filleted eleven-block sector

This is the most intricate construction in the tree and it is worth understanding as a case study in structured meshing, even though it is not yet wired into the optimizer.

The problem. Rounding the P_t corner *inside* the spoke block (the original approach, retired by §47) does not remove the fold—it moves it into whichever block carries the arc, where it becomes a cusp. The fix re-cuts the whole sector.

The construction. Per junction, two new blocks:

- `<j>_fillet_a`—a boundary layer along the fillet arc, above the ring circle;
- `<j>_fillet_b`—the wedge that crosses the ring circle, out to the ring's *far* side.

Plus the junction block (unchanged in shape, but its end cross-section is replaced by `fillet_a`'s inner edge) and the two ring blocks, re-cut. Seven blocks becomes eleven, fourteen whole-edge seams, closing at 3×10^{-14} mm.

Why two fillet blocks and not one. N is where the fillet block's inner edge crosses the ring circle. Above N that edge's partner is the junction block; below it, the ring. **One edge with two partners is a partial-edge seam**, and whole-edge single ownership is the module's whole safety net—so the block is split instead.

Why the cut must reach the ring's far boundary. This is a nice piece of reasoning. A cut stopping partway gives the ring free block's left edge two partners. Splitting *that* block gives its own right edge two partners, so the split propagates round the ring. And the block it propagates into is a **triangle**, because the inner edge is a concentric offset of an arc tangent to the ring circle, and is therefore tangent to every concentric circle (12.86° at the hub, 4.38° at the rim, measured). Carrying the cut all the way through splits the ring into two quadrilaterals and *terminates*. The price: the ring's radial node count becomes `n_thick`, so `n_collar_r` and `n_rim_r` are unused by this blocking.

The `entry` slope is negative on purpose. At `entry` = 0 the inner edge leaves the end cross-section *tangent* to the far flank—which is the junction block's own top edge—and three blocks meet at that node with 180° to share between two of them. Measured, the junction block's minimum scaled Jacobian is 0.0400 there against 0.4272 at the chosen slope.

Where it stands. As of §89–§92, no *mesh-validity* objection to wiring it in survives: the filleted and unfilleted meshes score the same 31-of-32 on a held-out draw against `MIN_SJ_TARGET`, and the one genome under the barrier is under it *harder* unfilleted (0.1298 versus 0.1775; barrier term 327.17 versus 18.20). What the fillet costs is **headroom, not crossings**—median minimum scaled Jacobian 0.338 against 0.745. Measured costs: 1.12× per objective evaluation (not the long-quoted 2–3×), with the one-time JIT trace at 4.13× and nineteen minutes. The two meshes disagree about the solved wheel by 37.97% of axle drop (linear, one phase) and 47.85% (SVK, eight phases). `tests/test_corner_singularity.py::test_nothing_wires_the_fillet_into_the_objective` holds the scope gate as a *check*, so it is a deliberate decision rather than an accident.

7 How it all fits together

7.1 One idea, applied at every level: write the thing once, derive the rest

The repository has a single organizing principle and applies it recursively.

- **In the element:** write the strain energy. The internal force is its gradient and the tangent is its Hessian, both by `jax.grad`/`jax.hessian`. No hand-derived tangent can drift.
- **In the contact law:** write the penalty potential. The contact force, the contact stiffness, *and the total ground reaction* are all derivatives of it. The reaction agreeing with an independent pressure quadrature is then evidence, not a tautology.
- **In the sensitivity:** the residual is already $\nabla\Pi$, so the adjoint is one `jax.grad` of one scalar. No separately coded sensitivity operator.
- **In the geometry:** `offset_band` is written once and serves both the exported outline (`n_across` = 1) and the mesh block (`n_across` = 6), so the meshed part and the exported part are the same surface by construction.
- **In the constants:** `MAX_ARRIVAL_DEG`, `MIN_FOLD_MARGIN_MM`, `TAPER_BREAKPOINTS` and `MIN_JUNCTION_BITE` each live in exactly one module, chosen so that both sides of the interpreter boundary can import it.

The consequences compound. Because the energy is written once, the linear and SVK kinematics share a material law and cannot disagree about it. Because the residual *is* a gradient, the Newton line search's slope is exact and a non-negative value diagnoses instability rather than noise. Because the geometry is closed-form and the connectivity is a compile-time constant, the whole pipeline from 14 genes to an axle drop is differentiable—which is what makes Stage 3 possible at all.

7.2 The second idea: name what is not differentiable

The tree is unusually disciplined about the boundary between smooth and discrete. Four things are frozen, and every one is *named* rather than tolerated:

1. **Flank orientation**—a topological fact about the genome; a step that flips it is a real discontinuity of the design space.
2. **Seam ownership**—a labelling, verified by `check_seams` to have made no difference.
3. **Fillet roots and refusals**—frozen from the eager build, refined by one Newton step.
4. `buckling`—the legacy Euler proxy, with an *asserted* zero gradient and a gate that fails the day it stops being zero.

And two things are named as *absent*: `R_hub` and `R_rim` have identically zero FEA gradient on the unfilleted mesh, because no fillet is meshed—so $\partial\,\text{coords}/\partial\,\text{gene}$ is zero before any solver is involved. This is not left to be discovered; `insensitive_genes` (`wheel_adjoint.py:744`) measures it on the Jacobian with tol $= 0.0$ ("these columns are not small, they are absent, and a tolerance would invite them to become small"), and the census is gated. Those two genes still carry a live gradient through the closed-form `fillet` and `fillet_cap` terms, which is the whole reason those terms exist.

7.3 The third idea: a measurement is not a number, it is a number plus its scope

Almost every constant in this tree carries the measurement that produced it, the sample it was measured on, and—repeatedly—the *previous* value and why it was wrong. The pattern that recurs:

- `HUB_CAP_THICKNESS_SHARE` was fitted tightly to a corner family that never binds.
- `MIN_FOLD_MARGIN_MM` is 0.1 and not 0 because zero admits six inverted meshes.
- `T1_REJECT` is 10^4 and not 1.0 because 1.0 deadlocked a probe for thirty-five minutes.
- `EMBED_ALLOWANCE_PER_SPOKE_MM2 = 3.03` is **deliberately not updated**, because the same computation has read 4.356, 3.032, 0.98 and 0.5317 on four genomes—"what is needed is the scaling law", and replacing the constant would only re-stale it on the next genome.
- `FILLET_LAYER_CLIFF_FACTOR = 0.45` is the *bottom edge* of an admissible band, because two axes improve monotonically across a flat third—and quoting the interior of a band is exactly the error a previous section made.

The `xfail_strict = true` setting in `pyproject.toml` is the same idea mechanised: an accepted finding is recorded as a strict `xfail` naming the section that decided it, so **the day the wheel changes enough for it to pass, the suite reopens the question by itself**.

7.4 Reading the tree

If you want to work in this repository, here is the order I would read in.

1. `CLAUDE.md`, then `PLAN.md`'s header block and the "Open arcs" table—that is the ranking and the rules.
2. `src/wheel_geometry.py` end to end. It is 436 lines and it is the vocabulary for everything else: Bézier, taper, offset band, fold margin, arrival.

3. `src/wheel_wheel.py`'s module docstring (lines 1–170). It explains why a structured partition of this wheel exists at all, what is and is not modelled, and what a seam mismatch would silently do.

4. `src/wheel_fem.py`'s module docstring, then `element_energy` and `_strain`. Sixty lines gets you the whole physics.

5. `src/wheel_adjoint.py`'s module docstring—the adjoint derivation and the "unnecessary / freezable / discrete" ordering for searches.

6. `src/wheel_objective.py`'s module docstring for the T1/T2/T3 economics, then jump to the term you care about.

7. Whichever `studies/study_*.py` docstring names the number you are about to quote. Every one of them says what it measured, on what, and what it does *not* establish.

And one habit the tree will reward: **before quoting any per-design number, check which genome it was measured on.** The provenance chain is `36aed36` (the GA/beam optimum, pinned forever) → `350f4c7` → `e4219f3` → `e126cc3` → `09e8188` (shipped). Several docstrings were written under earlier links and say so at the top. The banner in `PLAN.md` went stale for two promotions, and that fact is itself recorded as the warning.

Written from the source. Line references are against the `feature` branch as of 2026-08-29; `PLAN.md` sections referenced up to §92.